FLIGHT DELAY AND AIRPORT OPERATIONS INTELLIGENCE

M.Tech (Integrated)

in

School of Computer Science and Engineering

With

Specialization in Data Science

By

23MID0164 Chitraju Hyndavi
23MID0345 Vahnitha G

For

CSI3010

Data Warehousing and Data Mining

Under the guidance of

DEVIPRIYA A

School of Computer Science and Engineering



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

November, 2026

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE
	ABSTRACT	4
	LIST OF FIGURES	5
	LIST OF ABBREVIATIONS	6
	LIST OF TABLES	7
1	**INTRODUCTION**	8
	1.1 SYSTEM OVERVIEW	8
	1.2 OBJECTIVE	9
	1.3 APPLICATIONS	9
	1.4 LIMITATIONS	9
2	**SYSTEM ANALYSIS**	11
	2.1 EXISTING SYSTEM	11
	2.2 PROPOSED SYSTEM	11
	2.2.1 Benefits of Proposed System	12
3	**REQUIREMENT SPECIFICATION**	13
	3.1 HARDWARE REQUIREMENTS	13
	3.2 SOFTWARE REQUIREMENTS	13
4	**SYSTEM DESIGN SPECIFICATION**	15
	4.1 SYSTEM ARCHITECTURE	15

	4.2	DETAILED DESIGN	16
	4.3	DATABASE DESIGN	18
5	**RESULT AND CONCLUSION**		20
6	**APPENDICES**		30
	6.1	APPENDIX 1 - SAMPLE SOURCE CODE	30
	6.2	APPENDIX 2 - SCREEN SHOTS /OUTPUTs	30
7	**REFERENCES**		41
	7.1	LIST OF JOURNALS	41
	7.2	LIST OF WEBSITES (URLs)	41
	7.3	LIST OF BOOKS	42

ABSTRACT

Flight delays cost passengers and airlines time and money, and their causes are usually blamed on weather alone. This project builds a data warehouse from 5.8 million 2015 US flights and NOAA daily weather and uses it to measure how much weather changes delays and cancellations and to predict, at two realistic moments, whether a flight will arrive 15 or more minutes late. A delay is defined by the DOT rule, the same as in the base paper.

A star-schema warehouse in DuckDB supports OLAP, root-cause analysis and a study of how delays propagate along an aircraft's day. Eleven models are compared on a strict time-based split (train January to August, validate September, test November to December): LightGBM, XGBoost, Random Forest, Decision Tree, Logistic Regression, KNN, two Naive Bayes variants, a neural network and an ensemble, tuned by grid search and particle swarm optimisation and explained with SHAP.

Severe weather raises the share of delayed flights from 17.0% to 40.4% and the cancellation rate from 1.1% to 9.0%. At T-60 the best model reaches PR-AUC 0.645 (3.6 times the chance level) and day-ahead 0.360; weather features add 0.04 and 0.10 PR-AUC respectively. Compared with the base paper, the T-60 models are higher in accuracy and F2 at the same recall and in accuracy in every scenario at our own threshold, while the paper is ahead on the day-ahead task. The results are delivered in an interactive portal with a live flight predictor.

LIST OF FIGURES

SR. NO.	NAME OF THE FIGURE	PAGE NO.
1	The prediction moments studied: day-ahead, T-60 and (for comparison only) after take-off	8
2	System architecture	15
3	Star schema of the flight data warehouse	19
4	On-time share by scheduled departure hour	20
5	Weather effect on delay and cancellation, raw and controlled for airport and month	21
6	Departure delay against the inbound aircraft's arrival delay	21
7	T-60 model comparison by PR-AUC with bootstrap 95% intervals (boosted and classical models; the neural network and ensemble are in Table 5)	22
8	Precision-recall curves at T-60	23
9	LightGBM ablation: effect of weather, propagation and history features	24
10	SHAP summary of the T-60 model	25
11	Airport clusters (PCA projection)	26
12	Cancellation rules with the highest lift	27
13	Accuracy and F2 against the base paper in the three scenarios	28
14	Portal: overview page	31
15	Portal: T-60 performance page	32
16	Portal: live predictor with a real flight	33
17	Portal: day-ahead performance page	34
18	Portal: post-departure performance page	35
19	Portal: overall metric evaluation, scenarios	36
20	Portal: overall metric evaluation, models	37

21	Portal: precision-recall, ROC and lift curves and the threshold sweep	38
22	Portal: scorecard against the base paper	39
23	Portal: airports and cancellations	40

LIST OF ABBREVIATIONS

SR.NO.	ABBREVIATION
API	Application Programming Interface
AUC	Area Under the Curve
CART	Classification and Regression Tree
CI	Confidence Interval
CSV	Comma-Separated Values
DOT	(U.S.) Department of Transportation
DW	Data Warehouse
ETL	Extract, Transform, Load
GHCN-D	Global Historical Climatology Network - Daily
IATA	International Air Transport Association
KNN	K-Nearest Neighbours
MLP	Multilayer Perceptron
NAS	National Air System
NOAA	National Oceanic and Atmospheric Administration
OLAP	Online Analytical Processing
PR-AUC	Area Under the Precision-Recall Curve (average precision)
PSO	Particle Swarm Optimisation
ROC-AUC	Area Under the Receiver Operating Characteristic Curve

SHAP	SHapley Additive exPlanations
T-60	Sixty minutes before scheduled departure

LIST OF TABLES

SR.NO.	NAME OF THE TABLE	PAGE NO.
1	Feature sets of the three scenarios	17
2	Models and settings	17
3	Warehouse tables	19
4	Delay-cause shares for flights 15 or more minutes late	21
5	T-60 results on the November and December test flights (F1-tuned threshold)	22
6	Day-ahead results on the November and December test flights (F1-tuned threshold)	23
7	Post-departure results (F1-tuned threshold)	23
8	PR-AUC with and without weather features (tuned models)	23
9	Particle swarm tuning results	25
10	Airport clusters	26
11	Strongest non-redundant cancellation rules	26
12	Base paper versus ours at our F1-tuned threshold (percent; differences in points)	27
13	Base paper versus ours with the threshold lowered to match the paper's recall	28

CHAPTER 1
INTRODUCTION

1.1 SYSTEM OVERVIEW

Air travel in the United States moves several million passengers every week, and a flight that arrives late is one of the most visible failures of that system. The U.S. Department of Transportation (DOT) treats a flight as delayed when it arrives at least 15 minutes after its scheduled arrival time [12]. About one flight in five missed that mark in 2015, and the causes were routinely blamed on weather alone, even though airline operations, congestion at the airports and the way a late aircraft carries its delay into its next flights all contribute.

This project builds a complete analytics system around the 2015 DOT on-time performance data. The raw flight records (5,819,079 scheduled domestic flights) and the NOAA GHCN-Daily weather observations [14] are loaded into a star-schema data warehouse. On top of the warehouse the system performs online analytical processing (OLAP), quantifies how much weather really changes delays and cancellations, follows delays along each aircraft's day, and discovers patterns among airports and cancellations. Its central part is a prediction study: will a flight arrive 15 or more minutes late, predicted at two realistic moments. The first is the day before (day-ahead), using only the schedule, the airline, the airports and the weather. The second is one hour before scheduled departure (T-60), when the status of the inbound aircraft is also known.

The study compares eleven models, including gradient-boosted trees, classical learners, a neural network and an ensemble, under a strict time-based evaluation (train on January to August, tune on September, test on November and December). It is benchmarked against a published base paper on the same data [1]. The results are explained with SHAP and presented in an interactive local portal that includes a live flight predictor.

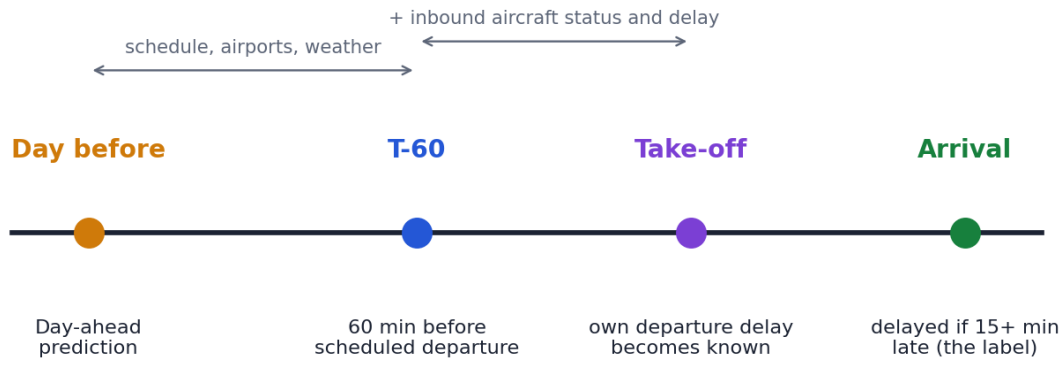


Figure 1: The prediction moments studied: day-ahead, T-60 and (for comparison only) after take-off

1.2 OBJECTIVE

The objectives of the project are the following:

- **O1 Data warehouse.** Design a dimensional model and an ETL pipeline that integrates flights, airlines, airports, routes and daily weather.
- **O2 OLAP analysis.** Roll-up, drill-down, slice, dice and pivot the data by time, airline and airport to find where and when delays occur.
- **O3 Root cause and weather impact.** Separate weather from other causes and measure its effect on delay and cancellation after controlling for airport, month, hour and airline.
- **O4 Delay propagation.** Follow each aircraft through its day using its tail number and measure how much of a delay is passed on, absorbed or amplified.
- **O5 Predictive mining.** Predict whether a flight arrives 15 or more minutes late at T-60 and day-ahead, compare algorithms fairly, quantify the contribution of weather and explain the predictions.
- **O6 Pattern discovery.** Cluster airports by their operating profile and mine association rules for flight cancellations.
- **O7 Base-paper comparison.** Compare accuracy, recall and F2 with the base paper [1] and state honestly where the results are higher and where they are not.
- **O8 Portal.** Deliver an interactive dashboard with a live predictor so that every result can be inspected and reproduced.

1.3 APPLICATIONS

- **Airline operations.** An alert one hour before departure lets an airline re-time crews, swap aircraft or prepare passenger notifications before the delay is visible at the gate.
- **Airport planning.** Day-ahead risk scores and the weather-sensitivity ranking of airports help schedule ground staff, de-icing and gate capacity.
- **Schedule design.** The turnaround-buffer analysis shows how long a planned turn must be to absorb an inbound delay, which supports more robust schedules.
- **Passenger information.** Early and explainable delay estimates reduce missed connections and improve customer communication.
- **Policy and research.** Separating weather, airline and air-system causes gives regulators and researchers a clearer picture of where delay minutes come from.

1.4 LIMITATIONS

- **Weather is observed, not forecast.** Same-day observed weather is used as a stand-in for a forecast, so the measured benefit of weather is an upper bound for a deployed system.

- **One year and one test period.** The data cover 2015 only and the models are scored on November and December; other seasons and years are untested.
- **October is excluded** from airport, weather and model work because its airport codes in the source file are numeric and could not be mapped to IATA codes.
- **Unknown inbound delay.** About 72% of flights have no known inbound arrival delay at T-60 (the inbound aircraft is still airborne, it is the first flight of the day, or it has not left yet), so the T-60 model relies on the inbound aircraft's departure delay and status.
- **Associations are not causes.** Cancellation rules and airport clusters describe patterns; clusters are soft and the rules reflect where carriers operate as well as how they behave.
- **Comparison with the base paper is approximate** because the splits, airport subsets and class balancing differ; the day-ahead task favours the paper.

CHAPTER 2
SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

Flight delay prediction has been studied with many techniques. Gui et al. combine aviation big data and machine learning to predict delays [2], Yazdi et al. train a deep network with the Levenberg-Marquardt algorithm [3], and Belcastro et al. use scalable data mining on large flight and weather collections [4]. The closest work to this project is the base paper by Schösser and Schönberger, who predict whether an arrival is delayed by 15 minutes or more on the same 2015 DOT data using Random Forest, XGBoost and a neural network [1].

The base paper evaluates three feature sets: static schedule and weather features only (set 1), plus the delay of the aircraft's previous flight (set 1+2), and plus the flight's own departure delay (set 1+3). It restricts the data to the 30 busiest airports (2.23 million flights), splits the records randomly into 75% for training and 25% for testing, balances the classes by undersampling the training set, tunes hyperparameters by grid search and particle swarm optimisation, and reports accuracy, recall and the F2 score. Its best model, XGBoost, reaches 69.5% accuracy and 68.2% recall without short-term features, 70.9% and 68.5% with the previous flight's delay, and 89.9% and 83.4% when the departure delay is added.

Existing systems of this kind have several weaknesses that this project addresses:

- A random split puts flights from the same day, and from the same aircraft rotation, in both the training and the test set, which makes the task easier than real forecasting.
- Only a subset of airports is used, so the results do not describe the whole network.
- Undersampling shifts the operating point towards high recall and low accuracy, and only accuracy, recall and F2 are reported; precision, ranking quality (ROC-AUC, PR-AUC) and uncertainty are not.
- Only three algorithms are compared and the effect of weather itself is not isolated.
- The work is a prediction study only: there is no data warehouse, no analysis of causes or propagation and no interface for users.

2.2 PROPOSED SYSTEM

The proposed system is an end-to-end analytics platform. Raw DOT and NOAA files are transformed by an ETL pipeline into a star-schema warehouse in DuckDB [10]. SQL analyses on the warehouse answer the descriptive questions (when, where and why delays occur, and how much weather matters). A leakage-controlled feature table is then built for each prediction moment and eleven models are trained on the early months and scored on the final months.

Predictions are explained with SHAP [7], the effect of weather is measured with paired ablations and bootstrap confidence intervals, and the results are compared with the base paper at two operating points. Everything is exposed through a local portal with separate pages for T-60, day-ahead and post-departure, an overall metric-evaluation page and a live predictor.

2.2.1 Benefits of Proposed System

- **Realistic evaluation.** A time-based split (train January to August, validate September, test November and December) means no test-month flight is seen during training.
- **Whole network.** All airports are used (113 airports with at least 5,000 flights are profiled), not only the 30 busiest.
- **Fair comparison.** Eleven models are trained with identical splits, class weights and threshold rules, and ranked by PR-AUC, the metric that stays honest when only 18% of flights are late.
- **Isolated weather effect.** Weather features are switched on and off for the tuned models, with paired bootstrap intervals, and the controlled effect on delay and cancellation is measured separately.
- **Leakage control.** An assertion fails the run if any feature uses information from after the prediction moment.
- **Explainability.** SHAP values, feature-group gains, ablations and subgroup checks show why the model predicts what it does.
- **Usability.** A portal lets a reviewer inspect every result and try the model on a real or hypothetical flight.

CHAPTER 3

REQUIREMENT SPECIFICATION

3.1 HARDWARE REQUIREMENTS

These are the hardware and software that support the system for smooth operation. The figures below are the configuration on which the project was developed and run.

Operating System

macOS (Darwin 25) on Apple silicon; the code also runs on Linux and Windows with Python 3.9 or later.

CPU

A multi-core processor; the experiments ran on a 10-core machine. The neural network uses the Apple Metal (MPS) GPU when available and falls back to the CPU.

RAM

16 GB or more is recommended. DuckDB is capped at 6 GB and the feature table of about five million rows is held in memory when models are trained.

Storage

About 4 GB of free disk space: the DuckDB warehouse (about 0.5 GB) plus a verified backup copy, the cached feature table and model predictions, and the result files.

Other requirements

A modern web browser (Google Chrome was used) for the portal. An internet connection is only needed to install the Python packages; the portal itself runs offline on localhost.

3.2 SOFTWARE REQUIREMENTS

The software description follows the different tools and libraries used for building, analysing and presenting the system.

1 Python 3.9 and virtual environment

All analysis and modelling code is written in Python 3.9 inside an isolated virtual environment (.venv) with the package versions pinned in requirements.txt, so the results can be reproduced.

2 DuckDB 1.4

DuckDB is an embeddable analytical (OLAP) database [10]. The star-schema warehouse is stored in a single file and queried with SQL, including window functions used for the aircraft-chain analysis. Every analysis script opens it read-only.

3 pandas, NumPy and PyArrow

pandas and NumPy handle tabular data and numerical work; PyArrow reads and writes

Parquet files used to cache the feature table.

4 scikit-learn 1.6

scikit-learn [8] provides Random Forest, Decision Tree, Logistic Regression, KNN, Naive Bayes, the preprocessing pipelines, K-Means, hierarchical clustering, PCA and the evaluation metrics.

5 LightGBM 4.6 and XGBoost 2.1

Gradient-boosted decision trees [5][6], the strongest model family in this study. LightGBM is the main model and also serves the live predictor.

6 PyTorch 2.8

PyTorch [11] is used for the embedding multilayer perceptron (MLP). Training runs on the Metal GPU when it is available.

7 SHAP, statsmodels, SciPy and mlxtend

SHAP [7] explains predictions; statsmodels fits the logistic regressions with clustered standard errors; SciPy provides rank statistics; mlxtend provides FP-Growth for association rules.

8 matplotlib and seaborn

Used for the static charts in the report and for the diagrams in Chapter 4.

9 d3.js and plain web technologies

The portal is written in HTML, CSS and JavaScript with d3 v7 for charts. A small Python server built on the standard library serves the files and a JSON API.

10 python-docx and headless Chrome

python-docx builds this report from the university template; headless Chrome captured the portal screenshots in Appendix 2.

CHAPTER 4

SYSTEM DESIGN SPECIFICATION

4.1 SYSTEM ARCHITECTURE

The system is organised as the layered pipeline shown in Figure 2. Source files enter an ETL layer that writes the warehouse; analysis modules read the warehouse, a feature layer prepares model inputs, the modelling and evaluation layer produces the results, and the portal and report present them.

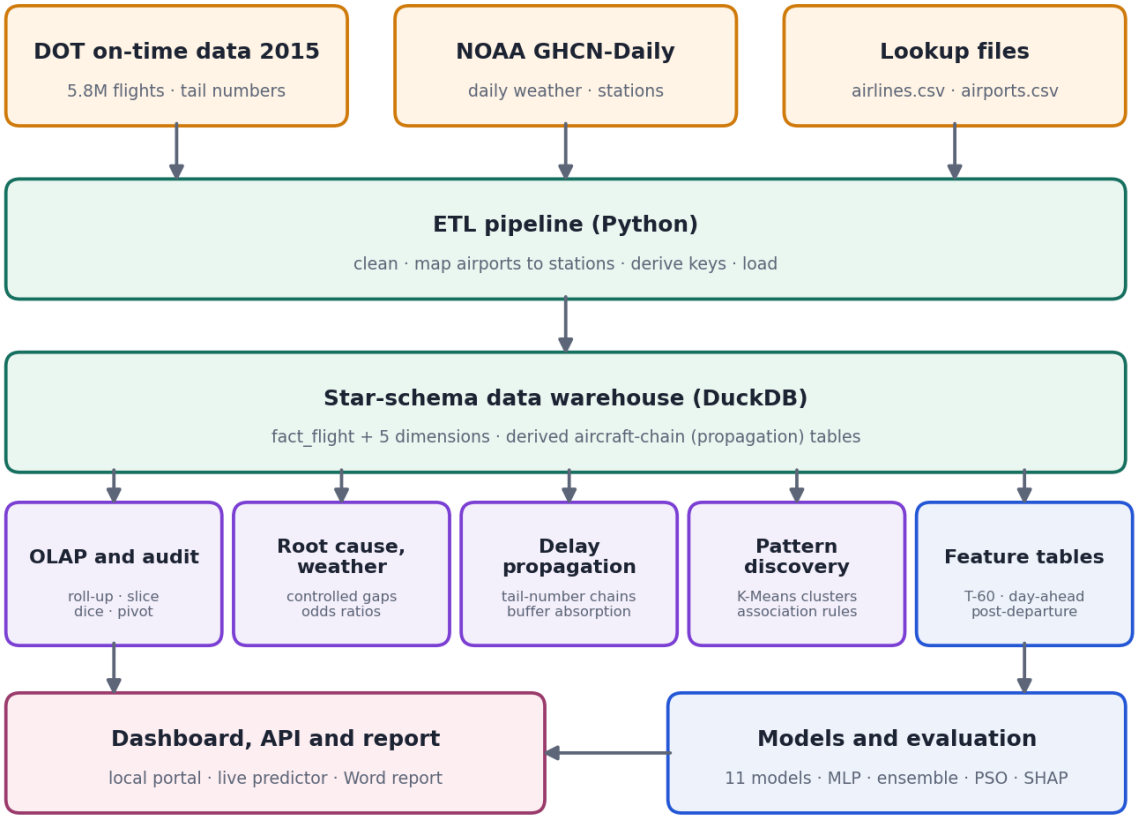


Figure 2: System architecture

- **Source layer.** The DOT on-time file (flights.csv, 5,819,079 rows), the airline and airport lookup files, the GHCN-Daily station list and the 37-million-row daily weather file.
- **ETL layer.** A Python pipeline (etl_pipeline.py) extracts the files, streams the weather file in chunks and keeps only quality-checked precipitation, snow, snow depth, temperature and wind readings, maps every airport to up to three suitable stations within 50 km, builds the dimensions and the fact table with surrogate keys, and writes CSV extracts that are loaded into DuckDB.
- **Warehouse layer.** One fact table and five dimensions (Section 4.3) plus two derived tables that hold the aircraft-chain features.

- **Analysis layer.** Scripts 00 to 06 produce the audit, the OLAP exports, the root-cause and weather analysis, the delay-propagation study, the airport clusters and the cancellation rules.
- **Feature and model layer.** A single SQL query builds the T-60 feature table; scenarios select feature subsets. Models are trained on the early months and their predictions are cached so that any result can be re-computed without retraining.
- **Presentation layer.** A local web portal (Python server with a JSON API and a d3 front end), this report and the screenshots.

4.2 DETAILED DESIGN

4.2.1 Data preparation and quality checks

The audit step (00_audit.py) verified that the warehouse has no duplicate keys, no orphan foreign keys, no flight that is both cancelled and diverted, and that every cancelled flight has a reason. It also found the October airport-code problem: the source uses numeric BTS airport identifiers for October, which the ETL mapped to the Unknown airport member by design, so October is kept for time-only OLAP and excluded from airport-level, weather and modelling work. Delayed is defined as an arrival delay of at least 15 minutes, matching the DOT rule and the base paper; flown flights exclude cancelled and diverted flights.

4.2.2 Weather classification

Each airport-day is classified from the observed daily measurements at the nearest station. Severe weather is snowfall above 25 mm, precipitation above 25 mm or average wind above 13.4 m/s; moderate weather is any snowfall, precipitation above 5 mm or wind above 8.9 m/s; everything else is clear. Missing measurements are treated as zero and airports with no station within the search radius are labelled unknown.

4.2.3 Delay propagation along an aircraft's day

For every flight with a tail number the legs of the aircraft are ordered by scheduled departure within each scheduled date using SQL window functions (LAG over all scheduled legs, so a cancelled leg breaks the chain instead of being skipped). Local clock times are converted to an absolute minute axis, which handles time zones and midnight rollover. A previous leg is accepted only if it belongs to the same aircraft and date, was flown, and arrived at this flight's origin; 98.9% of flown previous legs end at the next flight's origin, which confirms the chains are physically consistent. The analysis set has 3,917,656 flights; the subset whose previous arrival is known before the scheduled departure (3,693,636 flights) feeds the prediction features.

4.2.4 Prediction moments, features and leakage control

The T-60 feature table is built by one SQL query over the star schema and holds 5,231,130 flown flights from non-October months. Every column is known at prediction time: schedule and calendar attributes, airline, airports, route, observed weather at both ends (used as a stand-in for a forecast), the status of the inbound aircraft at T-60 (landed, airborne, not yet departed or no previous leg) with its departure and arrival delay where known, the planned turnaround, the leg number of the day, and historical delay rates of the route, airline and origin computed only from earlier training months. Day-ahead drops the six inbound-aircraft features; the post-departure scenario adds the flight's own departure delay and is used only to compare with the base paper. An assertion checks every feature against a forbidden list and verifies in the database that no inbound value uses an event later than T-60.

Table 1: Feature sets of the three scenarios

Scenario	Features	Information available
Day-ahead	37	Schedule, calendar, airline, airports, route, observed weather at both ends, historical delay rates
T-60	43	Day-ahead features + inbound aircraft status, departure and arrival delay, planned turnaround, leg number
Post-departure	44	T-60 features + the flight's own departure delay (known only after take-off)

4.2.5 Models

Eleven models are compared on identical splits. The training rows are Jan to Aug (3,836,893 flights), the validation rows are September (462,153) and the test rows are November and December (932,084; 17.95% delayed). Classes are weighted by $w = n / (2\, n_c)$ in every model that supports weights, and a single decision threshold per model is chosen to maximise F1 on September and applied unchanged to the test flights.

Table 2: Models and settings

Model	Training rows	Main settings
LightGBM	3.84 M	63 leaves, learning rate 0.05, min child samples 100, feature fraction 0.6, native categoricals, early stopping
XGBoost (tuned)	3.84 M	depth 8, learning rate 0.05, min child weight 50, column sample 0.6, native categoricals
Random Forest	1 M sample	150 trees, minimum leaf 50, sqrt features
Decision Tree	3.84 M	CART, depth 12 chosen on validation
Logistic Regression	3.84 M	one-hot (rare levels merged), scaled numerics, C = 1
KNN	200 k sample	k = 50, numeric features
Naive Bayes	3.84 M	Gaussian and categorical variants, balanced priors
Neural network	3.84 M	embeddings + 512-256-128 MLP, batch norm, dropout 0.25, class-weighted loss, AdamW, early stopping
Ensemble	-	mean of rank-normalised scores of LightGBM, XGBoost and the MLP

4.2.6 Evaluation protocol and tuning

PR-AUC (average precision) is the primary metric because it remains meaningful when

only 18% of flights are late; ROC-AUC, precision, recall, F1, F2 and accuracy are also reported, with accuracy marked as misleading (always predicting on-time already scores about 82%). Confidence intervals come from a paired Poisson bootstrap over test flights, and the boosted models were run with three seeds. Hyperparameters were first chosen by grid search on a 1-million-row sample against validation PR-AUC; as in the base paper, particle swarm optimisation (8 particles, 5 iterations, written for this project) was then run for XGBoost, LightGBM and the neural network.

4.2.7 Explainability, patterns and the portal

SHAP (exact TreeSHAP for LightGBM) gives per-flight and global explanations; feature-group gain, a depth-4 decision tree and subgroup scores complement them. Airports with at least 5,000 flights are clustered with K-Means (k chosen by silhouette among 3 to 6 and cross-checked with Ward clustering) on ten features, and cancellation association rules are mined exhaustively over conjunctions of up to three conditions with support, confidence and lift. The portal consists of a Python server that loads the saved T-60 LightGBM model and exposes /api/options, /api/predict and /api/sample, and a plain JavaScript front end (sidebar, tabs, sortable tables and d3 charts) with separate pages for T-60, day-ahead, post-departure, overall metric evaluation, data exploration and an About page.

4.3 DATABASE DESIGN

The warehouse is a star schema with one fact table at the grain of a scheduled flight leg and five dimensions; the airport and weather dimensions are role-played for the origin and the destination (Figure 3 and Table 3).

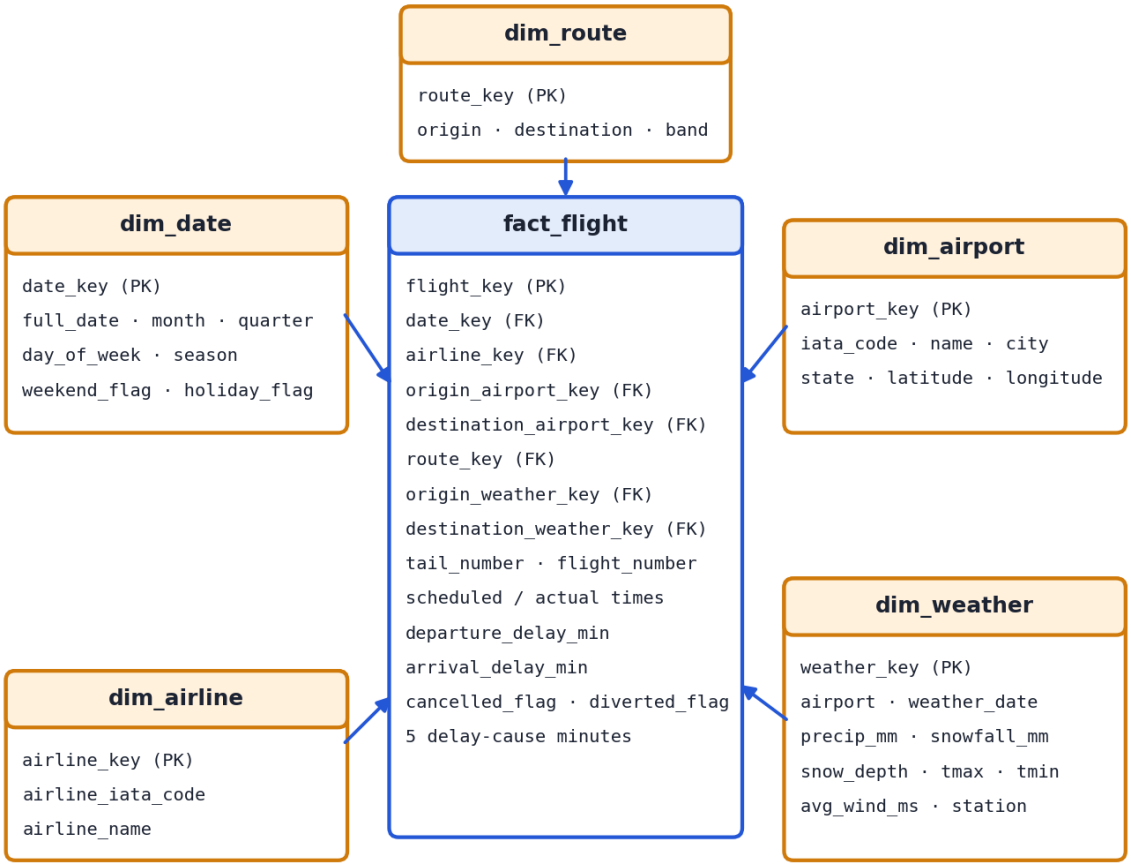


Figure 3: Star schema of the flight data warehouse

Table 3: Warehouse tables

Table	Grain / content	Key columns
fact_flight	One row per scheduled flight leg (5,819,079 rows)	flight_key; foreign keys to date, airline, route, origin and destination airport and weather; tail_number, flight_number; times; delay and cause minutes; cancelled and diverted flags
dim_date	One row per calendar day	date_key, full_date, month, quarter, day of week, season, weekend and holiday flags
dim_airline	One row per carrier	airline_key, IATA code, name
dim_airport	One row per airport	airport_key, IATA code, name, city, state, latitude, longitude
dim_route	One row per origin-destination pair	route_key, origin, destination, distance band, interstate flag
dim_weather	One row per airport per day	weather_key, precipitation, snowfall, snow depth, tmax, tmin, wind, station and distance
propagation_features	Derived: aircraft-chain features (analysis set)	previous leg arrival and departure delay, planned turn, leg number
propagation_t60	Derived: inbound status at T-60	inbound status, delays known at T-60, absolute-time checks

CHAPTER 5
RESULT AND CONCLUSION

5.1 RESULT

5.1.1 When and where flights are late (OLAP)

Delay builds through the day: the on-time share falls from 92.7% for 05:00 departures to 73.7% at 20:00, and evening departures are 2.4 times as likely to leave 15 or more minutes late as morning departures (27.0% against 11.3%). The worst months are February (76.65% on time, 4.78% cancelled) and June (76.52%), the best is September (87.00%). Saturday is the best weekday (84.01% on time) and Monday has the highest cancellation rate (2.43%). Hawaiian Airlines (88.67%), Alaska (86.96%) and Delta (86.44%) are the most punctual large carriers; Spirit (70.29%) and Frontier (73.84%) are the least, and American Eagle cancels 5.1% of its flights, three times the network rate.

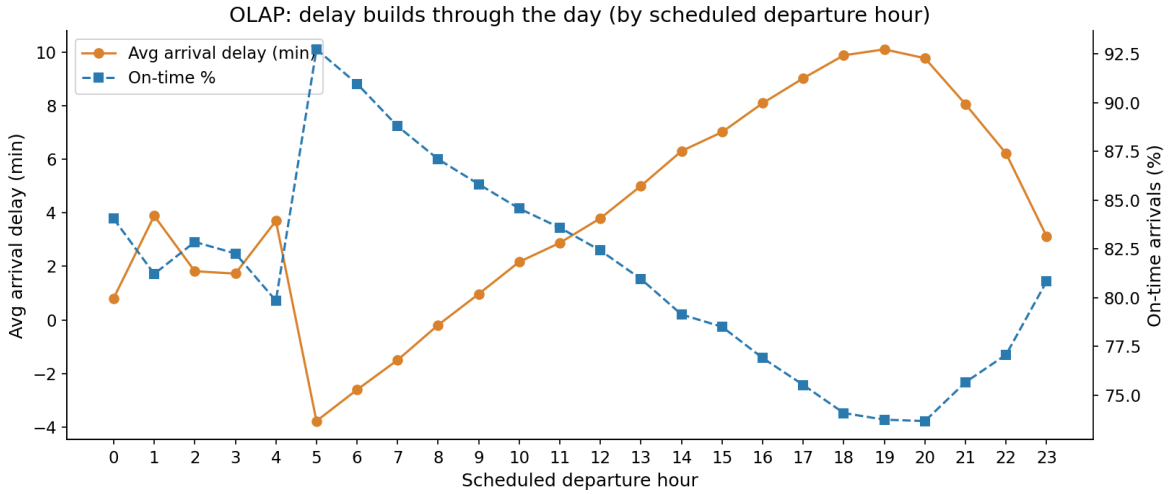


Figure 4: On-time share by scheduled departure hour

5.1.2 Weather and the causes of delay

Severe origin weather raises the share of delayed flights from 17.0% on clear days to 40.4% (moderate: 27.6%) and the cancellation rate from 1.1% to 9.0%. Controlling for airport and month leaves the gap unchanged (+23.4 percentage points, about +26 minutes of arrival delay), and adjusting for airport, month, hour and airline gives odds ratios of 3.60 for a delay and 7.70 for a cancellation in severe weather. Weather causes 54.8% of all cancellations (47,691 of 87,076). Among delayed flights, late aircraft account for 39.8% of delay minutes, airline operations for 32.2%, the air system for 22.9% and weather for only 4.9%; DOT's weather code captures only about a fifth of the extra delay on severe-weather days because air-system and late-aircraft delays rise as well.

Table 4: Delay-cause shares for flights 15 or more minutes late

Cause	Delay minutes	Share
Late aircraft	24,961,931	39.8%
Airline	20,172,956	32.2%
Air system (NAS)	14,335,762	22.9%
Weather	3,100,233	4.9%
Security	80,985	0.1%

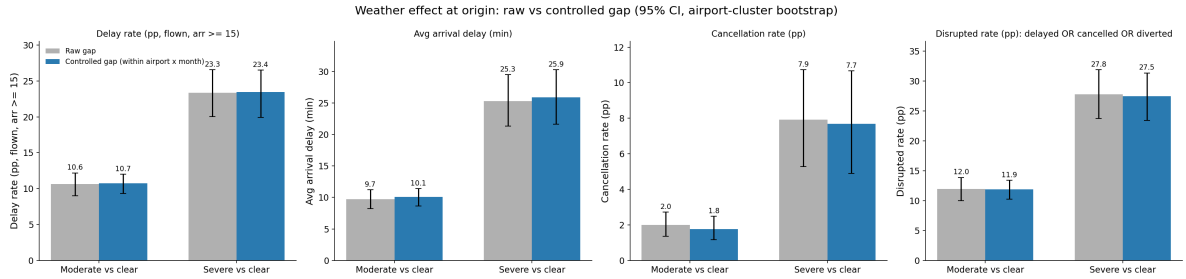


Figure 5: Weather effect on delay and cancellation, raw and controlled for airport and month

5.1.3 Delay propagation

Of all departure delays of 15 minutes or more in the analysis set, 60.8% follow an inbound flight that arrived at least 15 minutes late. When the inbound aircraft is 30 to 59 minutes late, 83.2% of the next departures leave 15 or more minutes late, against 16.5% when the inbound arrival was only 0 to 14 minutes late. The mean departure delay rises from 5.0 minutes on an aircraft's first leg to 16.4 minutes on its seventh. A planned turnaround of 90 minutes or more absorbs about three quarters of delays under an hour, while a turn under 45 minutes absorbs nothing. SkyWest and Hawaiian absorb the most inbound delay, while United, JetBlue and Spirit amplify it.

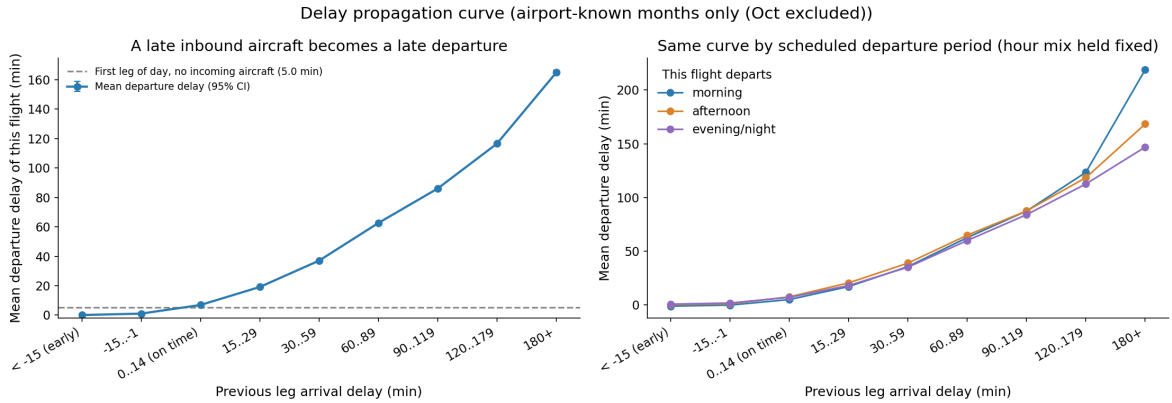


Figure 6: Departure delay against the inbound aircraft's arrival delay

5.1.4 Prediction at T-60 and day-ahead

At T-60 the ensemble of LightGBM, XGBoost and the neural network ranks flights best (PR-AUC 0.645, ROC-AUC 0.824, a lift of 3.6 times over the 18% chance level), closely followed by tuned XGBoost and LightGBM; the three are statistically close and the ensemble's lead is small. The neural network (0.634) trails the boosted trees, as is typical for tabular data, but

beats Random Forest. Naive Bayes and KNN are weakest. Day-ahead is much harder (best PR-AUC 0.360, 2.0 times chance) because nothing is known about the aircraft's earlier legs.

Table 5: T-60 results on the November and December test flights (F1-tuned threshold)

Model	PR-AUC	ROC-AUC	Accuracy	Precision	Recall	F1	F2
Ensemble	**0.645**	**0.824**	**87.0%**	**72.4%**	**44.3%**	**0.549**	**0.480**
XGBoost (tuned)	0.641	0.821	85.2%	60.0%	52.7%	0.561	0.541
XGBoost (ordinal)	0.639	0.819	85.1%	59.4%	53.0%	0.560	0.542
LightGBM	0.639	0.820	84.8%	58.4%	53.8%	0.560	0.547
Neural network	0.634	0.817	85.9%	64.1%	48.7%	0.554	0.512
Random Forest	0.619	0.813	84.9%	59.3%	51.2%	0.550	0.527
Decision Tree	0.604	0.793	85.7%	64.7%	45.3%	0.533	0.482
Logistic Regression	0.550	0.781	82.8%	52.1%	49.4%	0.507	0.499
KNN	0.478	0.740	82.4%	51.2%	41.7%	0.460	0.433
NB (categorical)	0.459	0.732	82.4%	51.2%	36.9%	0.429	0.391
NB (Gaussian)	0.362	0.720	71.8%	34.0%	60.7%	0.436	0.524

Table 6: Day-ahead results on the November and December test flights (F1-tuned threshold)

Model	PR-AUC	ROC-AUC	Accuracy	Precision	Recall	F1	F2
Ensemble	**0.360**	**0.703**	**75.0%**	**34.7%**	**44.5%**	**0.390**	**0.421**
XGBoost (tuned)	0.356	0.700	69.8%	30.9%	55.3%	0.397	0.478
LightGBM	0.354	0.699	70.3%	31.2%	54.1%	0.396	0.472
Random Forest	0.342	0.693	68.3%	29.8%	56.7%	0.391	0.480
XGBoost (ordinal)	0.341	0.687	65.8%	28.5%	59.7%	0.385	0.489
Neural network	0.337	0.689	73.4%	32.5%	44.8%	0.377	0.416
Logistic Regression	0.320	0.670	69.8%	29.7%	49.6%	0.371	0.437
NB (Gaussian)	0.288	0.653	49.5%	22.8%	75.8%	0.350	0.517
KNN	0.285	0.623	71.8%	28.5%	37.7%	0.325	0.354
Decision Tree	0.285	0.637	73.1%	29.7%	36.6%	0.328	0.350
NB (categorical)	0.271	0.628	74.0%	29.1%	30.9%	0.300	0.305

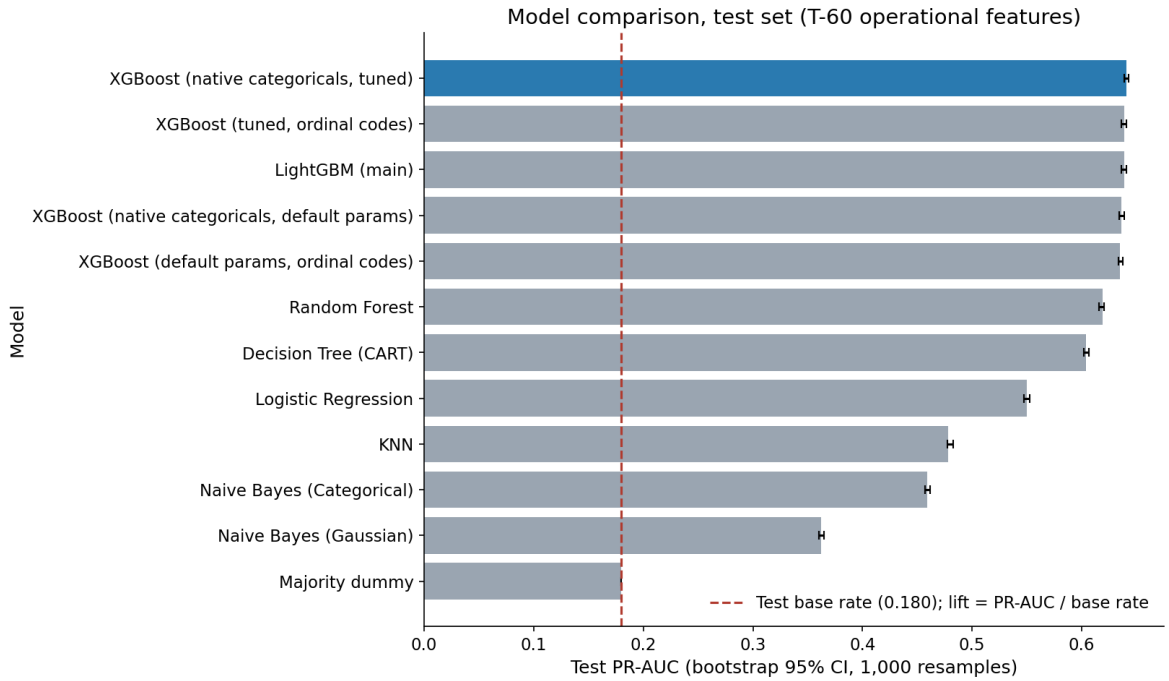


Figure 7: T-60 model comparison by PR-AUC with bootstrap 95% intervals (boosted and classical models; the neural network and ensemble are in Table 5)

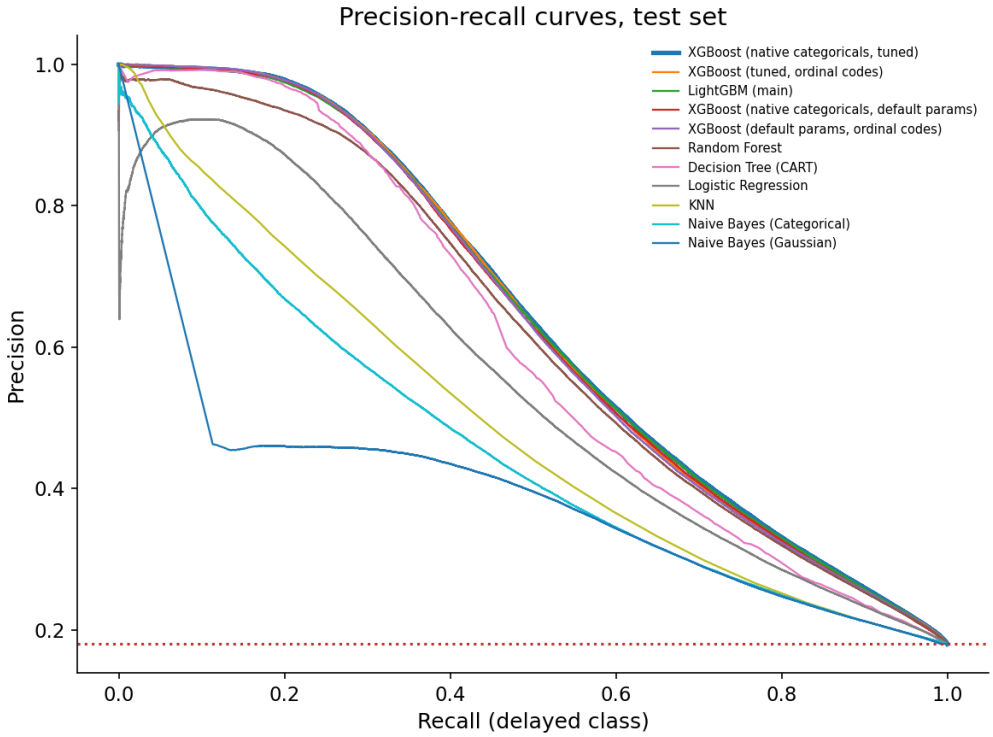


Figure 8: Precision-recall curves at T-60

5.1.5 Post-departure scenario

When the flight's own departure delay is added, ranking quality rises sharply (XGBoost PR-AUC 0.880, ROC-AUC 0.944, 4.9 times chance), but this number is known only after take-off, so the scenario exists to compare with the base paper and to show how much a single piece of information is worth.

Table 7: Post-departure results (F1-tuned threshold)

Model	PR-AUC	ROC-AUC	Accuracy	Precision	Recall	F1	F2
XGBoost (tuned)	0.880	0.944	93.1%	84.4%	75.8%	0.799	0.773
LightGBM	0.879	0.943	93.0%	83.9%	75.9%	0.797	0.773

5.1.6 Effect of weather on prediction

Switching the weather features off and on for the tuned models shows how much weather contributes. The effect is large when nothing else is known and smaller, but still clearly non-zero, when the inbound aircraft is known; all four intervals exclude zero. Weather also accounts for 11.7% of LightGBM's gain, behind the inbound aircraft's status and delay (42.3%) and airline, airport and route (24.3%). The weather used is observed same-day weather, so these gains are upper bounds for a real forecast.

Table 8: PR-AUC with and without weather features (tuned models)

Scenario	Model	Without	With	Gain (95% interval)
T-60	XGBoost	0.599	0.641	+0.042 [0.041, 0.043]
T-60	LightGBM	0.599	0.642	+0.043 [0.043, 0.044]
Day-ahead	XGBoost	0.247	0.344	+0.097 [0.095, 0.099]

Day-ahead	LightGBM	0.244	0.347	+0.103 [0.101, 0.105]

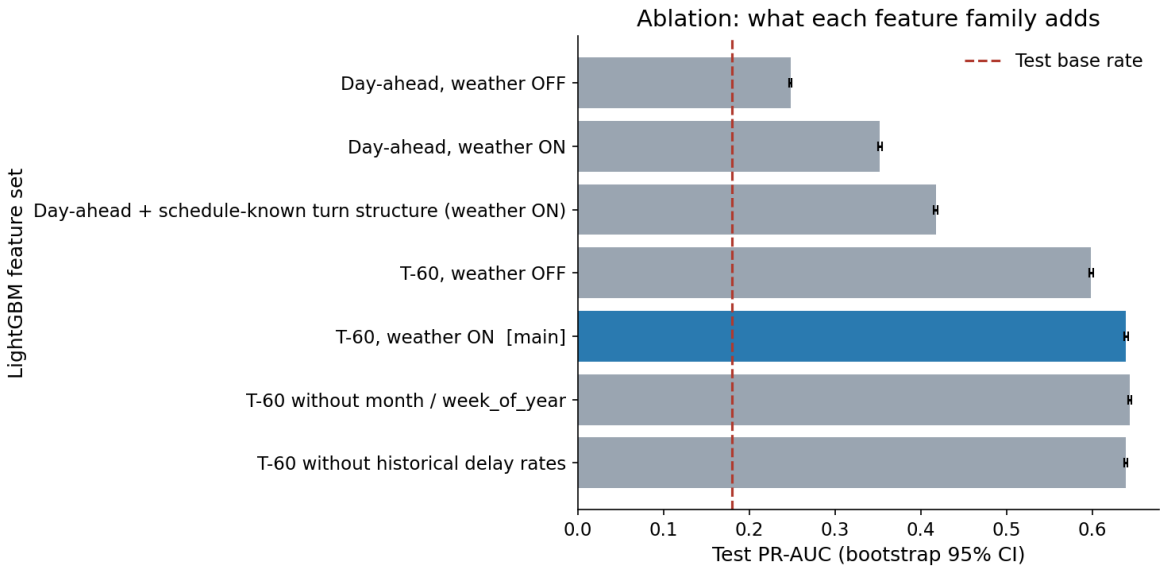


Figure 9: LightGBM ablation: effect of weather, propagation and history features

5.1.7 Explainability

The mean absolute SHAP values show that the most influential features at T-60 are the inbound aircraft's departure delay, the time of year, destination precipitation and the inbound status. A late inbound aircraft pushes the risk up sharply, while a long planned turnaround and early departure hours lower it.

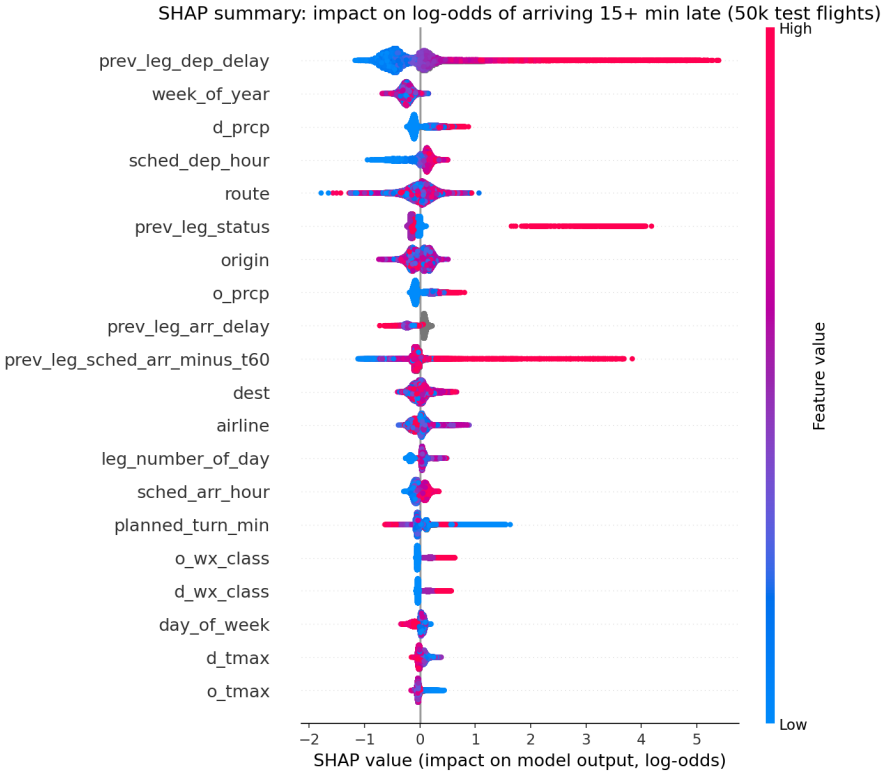


Figure 10: SHAP summary of the T-60 model

5.1.8 Hyperparameter tuning

Particle swarm tuning, run on a 600,000-row training sample, improved T-60 LightGBM by 0.004 PR-AUC and changed the other models by less than that or made them slightly worse, because settings chosen on a sample did not carry over to full-data training. The earlier grid-searched configurations therefore remain the reference results.

Table 9: Particle swarm tuning results

Scenario	Model	Tuned values	Before	After
T-60	XGBoost	max_depth 9 · learning_rate 0.0546 · min_child_weight 40 · colsample_bytree 0.48 · gamma 0.106 · subsample 0.989	0.639	0.641
T-60	LightGBM	num_leaves 164 · learning_rate 0.05 · min_child_samples 674 · feature_fraction 0.4 · lambda_l2 0.01 · bagging_fraction 1	0.639	0.642
Day-ahead	XGBoost	max_depth 9 · learning_rate 0.05 · min_child_weight 55.7 · colsample_bytree 0.4 · gamma 1.02 · subsample 1	0.341	0.344
Day-ahead	LightGBM	num_leaves 78 · learning_rate 0.0984 · min_child_samples 2000 · feature_fraction 0.767 · lambda_l2 0.815 · bagging_fraction 1	0.354	0.347
T-60	Neural network	width 676 · drop 0.103 · lr 0.00757 · wd 8.82e-05 · bs 1024	0.634	0.625
Day-ahead	Neural network	width 781 · drop 0.199 · lr 0.00383 · wd 0.000212 · bs 1024	0.337	0.333

5.1.9 Airport clusters and cancellation rules

K-Means on 113 airports gives three groups: large hubs with the longest delays and the greatest sensitivity to adverse weather, mid-size airports that are the most punctual, and smaller airports with the highest cancellation rate and weather exposure. The clusters are soft (silhouette 0.24) but stable under resampling (mean adjusted Rand index 0.88) and agree with Ward clustering (0.82). Exhaustive rule mining over 5.30 million flights (87,076 cancelled, base rate 1.64%) shows that cancellations are concentrated where severe weather, snow, high wind and freezing temperatures combine, with lifts of 20 to 28; the regional carriers MQ and EV recur in the strongest rules. These are associations, not causes.

Table 10: Airport clusters

Cluster	Airports	Delayed	Cancelled	Weather gap (pp)	Examples
Mid-size, punctual	35	15.9%	0.94%	7.6	PHX, SEA, SLC, FLL, SAN
Large hubs, congested	24	21.0%	1.77%	16.9	ATL, ORD, DFW, DEN, LAX
Smaller, weather-exposed	54	18.1%	2.33%	10.7	BNA, CLE, RDU, MKE, IND

Table 11: Strongest non-redundant cancellation rules

If the flight has	Flights	Cancelled	Share	Lift
airline=MQ & o_snow & d_severe_wx	2,795	1,274	45.6%	27.7x
airline=MQ & o_severe_wx & d_severe_wx	2,576	1,146	44.5%	27.1x
o_severe_wx & o_high_wind	3,542	1,568	44.3%	26.9x
o_snow & o_high_wind	3,359	1,469	43.7%	26.6x
month=February & airline=MQ & o_severe_wx	2,825	1,135	40.2%	24.4x
month=February & airline=MQ & d_severe_wx	2,816	1,100	39.1%	23.8x

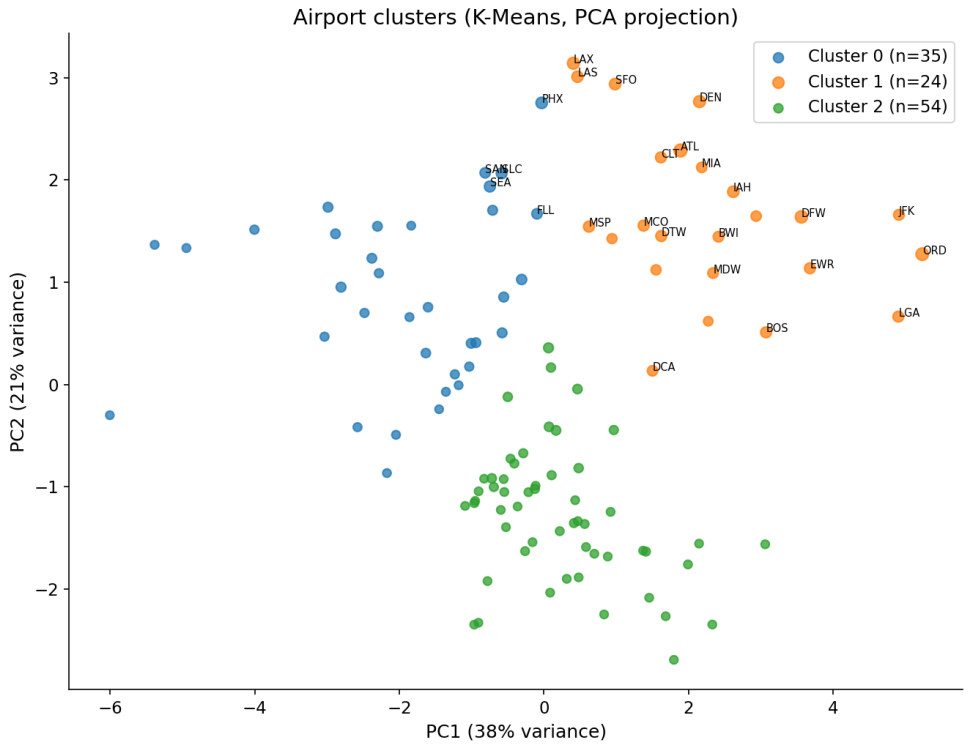


Figure 11: Airport clusters (PCA projection)

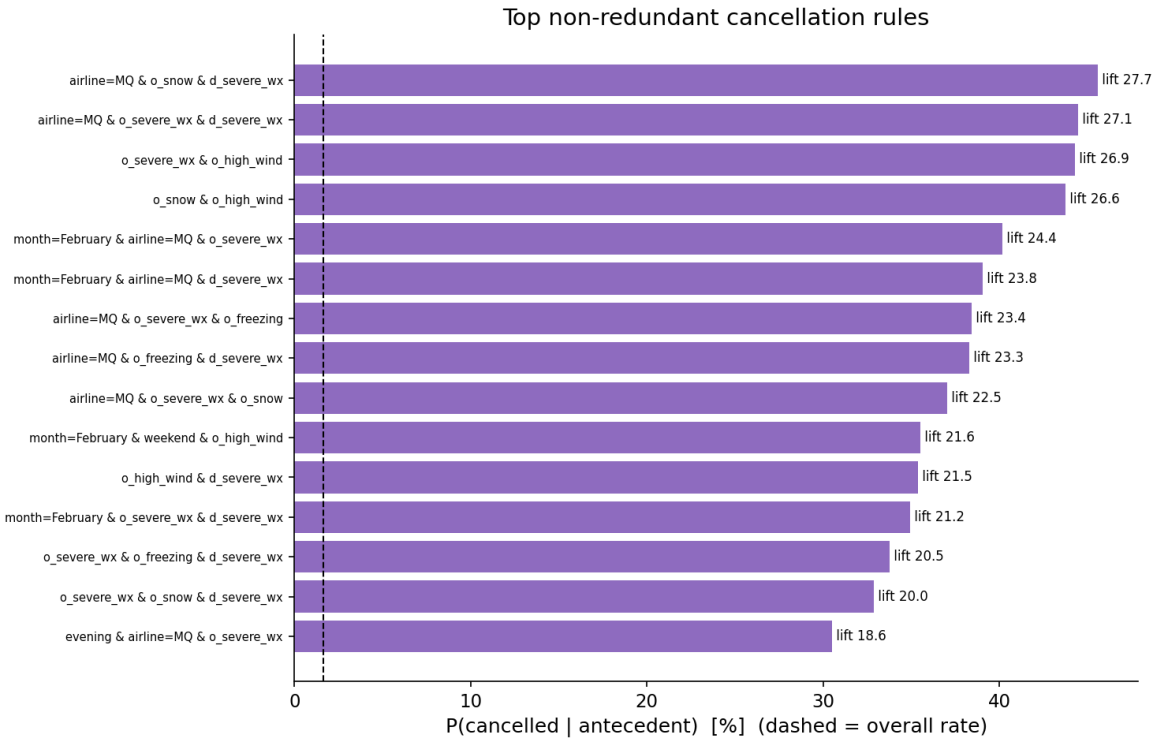


Figure 12: Cancellation rules with the highest lift

5.1.10 Comparison with the base paper

Both studies use the same 2015 data and the same 15-minute rule, and both find tree ensembles ahead of the neural network and both find that the inbound aircraft's delay helps. The comparison is approximate because the base paper uses a random split on 30 airports with undersampled training, so Table 12 shows our models at their own F1-tuned threshold and Table

13 at a lowered threshold whose recall is close to the paper's. At our own threshold we exceed the paper's accuracy in every scenario (by 0.3 points day-ahead, 14.3 at T-60 and 3.2 post-departure for XGBoost) but have lower recall and F2, as expected when the threshold favours precision. At matched recall the T-60 models are clearly higher (ensemble 79.2% accuracy and 61.0% F2 against 70.9% and 58.1%), the post-departure models are level on F2 (80.0% against 80.3%), and the paper is ahead on the day-ahead task.

Table 12: Base paper versus ours at our F1-tuned threshold (percent; differences in points)

System	Acc.	Prec.	Recall	F2	Δ Acc.	Δ Rec.	Δ F2
Day-ahead							
Paper XGBoost	69.5	-	68.2	57.2	-	-	-
Ours: LightGBM	70.3	31.2	54.1	47.2	+0.8	-14.1	-10.0
Ours: XGBoost	69.8	30.9	55.3	47.8	+0.3	-12.9	-9.4
Ours: Random Forest	68.2	29.8	56.6	48.0	-1.2	-11.6	-9.2
Ours: Neural network	73.4	32.5	44.8	41.6	+3.9	-23.4	-15.6
Ours: Ensemble	75.0	34.7	44.5	42.1	+5.5	-23.7	-15.1
T-60							
Paper XGBoost	70.9	-	68.5	58.1	-	-	-
Ours: LightGBM	84.8	58.4	53.8	54.7	+13.9	-14.7	-3.4
Ours: XGBoost	85.2	60.0	52.7	54.0	+14.3	-15.8	-4.1
Ours: Random Forest	84.9	59.3	51.2	52.6	+14.0	-17.3	-5.5
Ours: Neural network	85.9	64.1	48.7	51.2	+15.0	-19.8	-6.9
Ours: Ensemble	87.0	72.4	44.3	48.0	+16.1	-24.2	-10.1
Post-departure							
Paper XGBoost	89.9	-	83.4	80.3	-	-	-
Ours: LightGBM	93.0	83.9	75.9	77.3	+3.1	-7.5	-3.0
Ours: XGBoost	93.1	84.4	75.8	77.3	+3.2	-7.7	-3.0

Table 13: Base paper versus ours with the threshold lowered to match the paper's recall

System	Acc.	Prec.	Recall	F2	Δ Acc.	Δ Rec.	Δ F2
Day-ahead							
Paper XGBoost	69.5	-	68.2	57.2	-	-	-
Ours: Ensemble	62.9	27.9	67.3	52.5	-6.6	-0.9	-4.7
Ours: XGBoost	55.9	25.4	75.4	54.1	-13.6	+7.2	-3.1
T-60							
Paper XGBoost	70.9	-	68.5	58.1	-	-	-
Ours: Ensemble	79.2	44.7	67.1	61.0	+8.3	-1.4	+2.9
Ours: XGBoost	72.4	36.9	75.3	62.3	+1.5	+6.8	+4.2
Post-departure							
Paper XGBoost	89.9	-	83.4	80.3	-	-	-
Ours: LightGBM	88.4	62.9	85.8	80.0	-1.5	+2.4	-0.3
Ours: XGBoost	87.9	61.7	86.3	79.9	-2.0	+2.9	-0.4

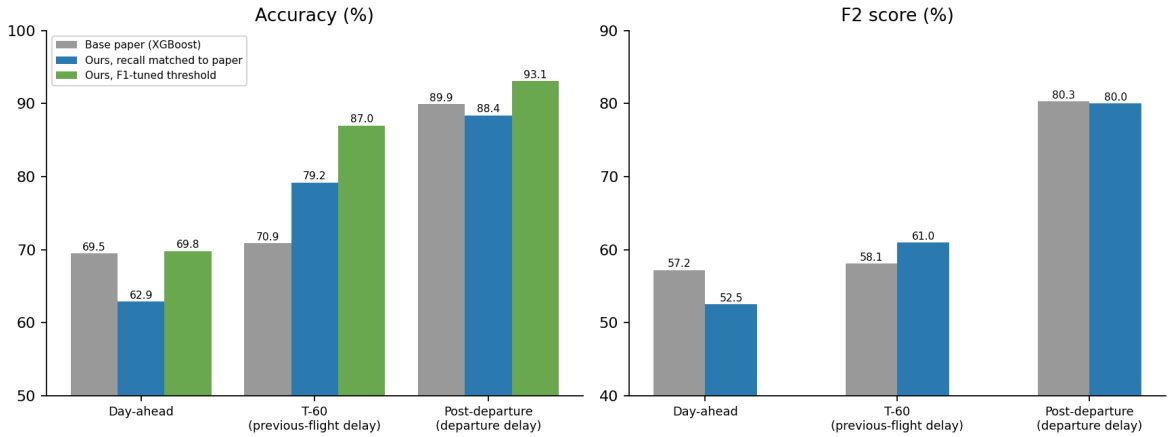


Figure 13: Accuracy and F2 against the base paper in the three scenarios

5.1.11 The portal

The local portal (Appendix 2) has a page for each prediction moment, an overall metric-evaluation page with confidence intervals, a model-by-metric heatmap, precision-recall, ROC and lift curves and a threshold sweep, a data-exploration page, and a live predictor. The predictor returns a probability, the alert verdict relative to the September-tuned threshold and the six features that pushed the estimate, and can load a real November or December flight to compare the prediction with what actually happened.

5.2 CONCLUSION

This project built a data warehouse from 5.8 million 2015 US flights and NOAA weather and used it to answer two questions. First, weather matters: severe weather at the origin multiplies the delay rate by 2.4 (17.0% to 40.4%) and the cancellation rate by about 8, and the effect survives controls for airport, month, hour and airline. Second, early prediction is feasible and its quality depends mainly on what is known: the best PR-AUC rises from 0.36 day-ahead to 0.64 at T-60 and 0.88 once the departure delay is known, while weather features add 0.10 and 0.04 PR-AUC at the two operational moments. A late inbound aircraft is the strongest single signal, and planned turnaround buffers of 90 minutes or more absorb most delays under an hour.

Against the base paper, the results are higher where the comparison is like for like: the T-60 models exceed the paper's accuracy and F2 at the same recall, and at our own threshold accuracy is higher in every scenario. The paper remains ahead on the day-ahead task and on recall at our F1 threshold, and the comparison is approximate because of different splits, airport subsets and class balancing. The evaluation here is stricter (time-based split, all airports, confidence intervals, PR-AUC and ROC-AUC reported) and the study is broader (warehouse, root-cause analysis, propagation, clustering, association rules and an interactive portal).

Future work includes using real weather forecasts instead of observed weather, extending the data to several years and to international flights, adding crew and airport-capacity information,

calibrating the probabilities for operational use, deploying the predictor as a service, and training a day-ahead model that can be queried in the portal.

CHAPTER 6

APPENDICES

6.1 APPENDIX 1 - SAMPLE SOURCE CODE

The listings below are short excerpts of the project code (full scripts are in the mining/ folder).

A. Leakage assertion (excerpt)

```
def assert_no_leakage(features, verbose=True, allow_rebuild=True):
    feats = list(features)
    bad = [c for c in feats if c in FORBIDDEN_EXACT or (FORBIDDEN_PATTERN.search(c) and c
          not in ALLOWED_PREV)]
    assert not bad, f"LEAKAGE: forbidden columns in feature matrix: {bad}"
    unknown = [c for c in feats if c not in ALL_FEATURES]
    assert not unknown, f"Features outside the approved whitelist: {unknown}"
    # database check: no inbound-aircraft value may use an event later than T-60
    r = con.execute("SELECT sum((prev_leg_arr_delay IS NOT NULL AND (prev_arr_abs > t60_abs
                      OR prev_leg_status <> 'landed'))::int), ... FROM
                            propagation_t60").fetchone()
    assert all(v == 0 for v in r), f"LEAKAGE: inbound feature uses an event after T-60: {r}"
```

B. Particle swarm optimiser (mining/pso.py)

```
def pso(objective, space, n_particles=8, n_iter=5, seed=42, w=0.7, c1=1.5, c2=1.5,
      log_path=None):
    rng = np.random.default_rng(seed); d = len(space)
    pos = rng.random((n_particles, d)); vel = rng.normal(0, 0.1, (n_particles, d))
    pbest, pscore = pos.copy(), np.full(n_particles, -np.inf); gbest, gscore =
          pos[0].copy(), -np.inf
    for it in range(n_iter + 1):
        for i in range(n_particles):
            p = decode(pos[i], space); s = float(objective(p))
            if s > pscore[i]: pscore[i], pbest[i] = s, pos[i].copy()
            if s > gscore: gscore, gbest = s, pos[i].copy()
        r1, r2 = rng.random((n_particles, d)), rng.random((n_particles, d))
        vel = w * vel + c1 * r1 * (pbest - pos) + c2 * r2 * (gbest - pos); pos =
              np.clip(pos + vel, 0, 1)
    return decode(gbest, space), gscore
```

C. Aircraft-chain window function (excerpt of the propagation SQL)

```
SELECT *, row_number() OVER p AS leg_number,
       lag(flown) OVER p AS prev_flown, lag(d) OVER p AS prev_dest, lag(arr_abs) OVER p
            AS prev_arr_abs,
       lag(dd) OVER p AS prev_dep_delay, lag(ad) OVER p AS prev_arr_delay
FROM c
WINDOW p AS (PARTITION BY tail_number, date_key ORDER BY sdep, flight_number, flight_key)
```

D. Prediction API (excerpt of dashboard/server.py)

```
def score(X):
    X = X[FEATS].copy()
    for c in CAT: X[c] = pd.Categorical(X[c])
    with lock: prob = float(bst.predict(X)[0]); con = bst.predict(X, pred_contrib=True)[0]
    top = np.argsort(-np.abs(con[:-1]))[:6]                    # six largest SHAP
          contributions
    return dict(probability=prob, threshold=THR, predicted_delayed=prob >= THR,
          base_rate=BASE_RATE, lift=prob / BASE_RATE,
                drivers=[...])
```

6.2 APPENDIX 2 - SCREEN SHOTS / OUTPUTS

The following screenshots show the portal (light theme) captured with headless Chrome. The predictor screenshot shows a real flight from the November and December test period together with what actually happened.

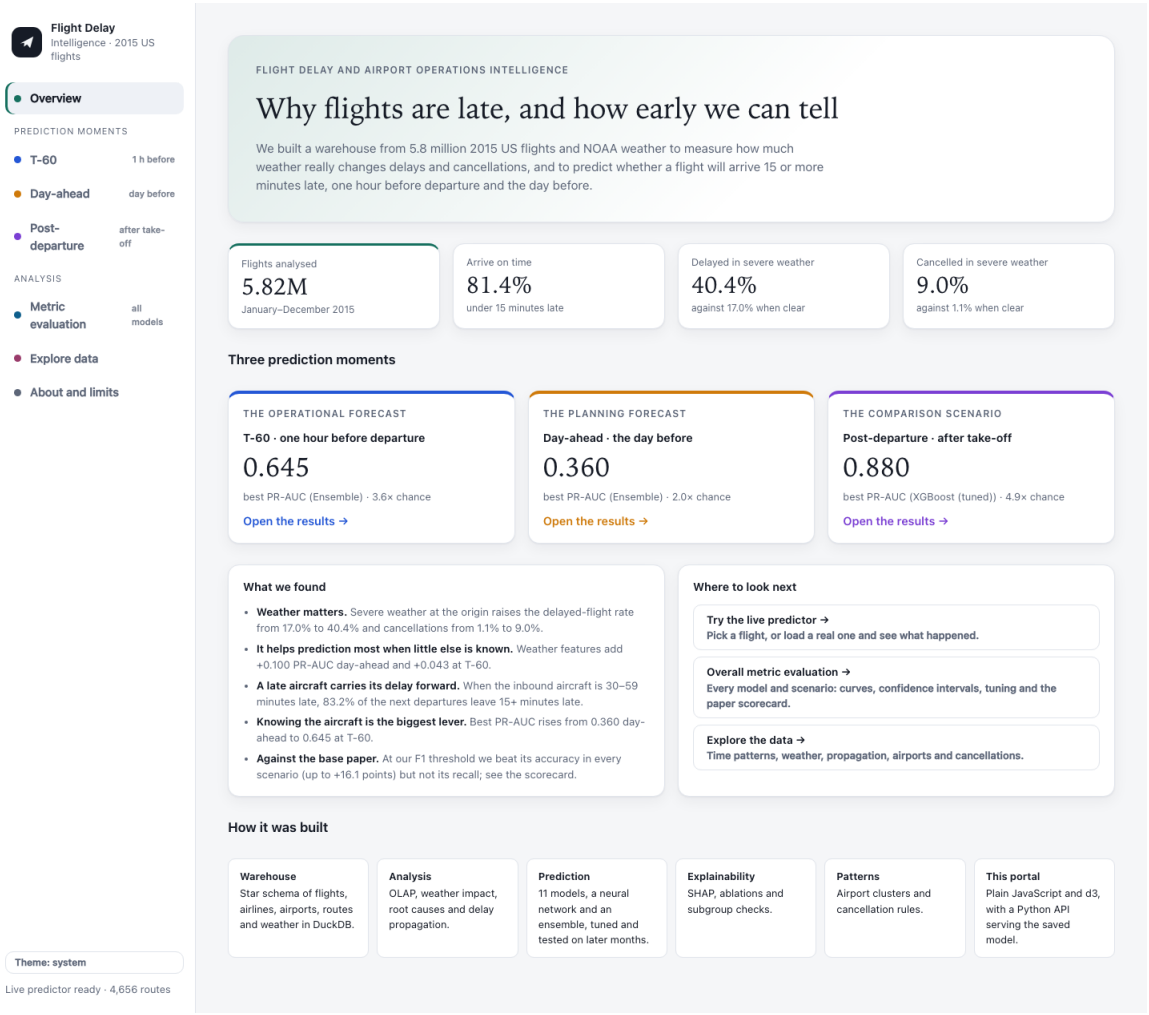


Figure 14: Portal: overview page

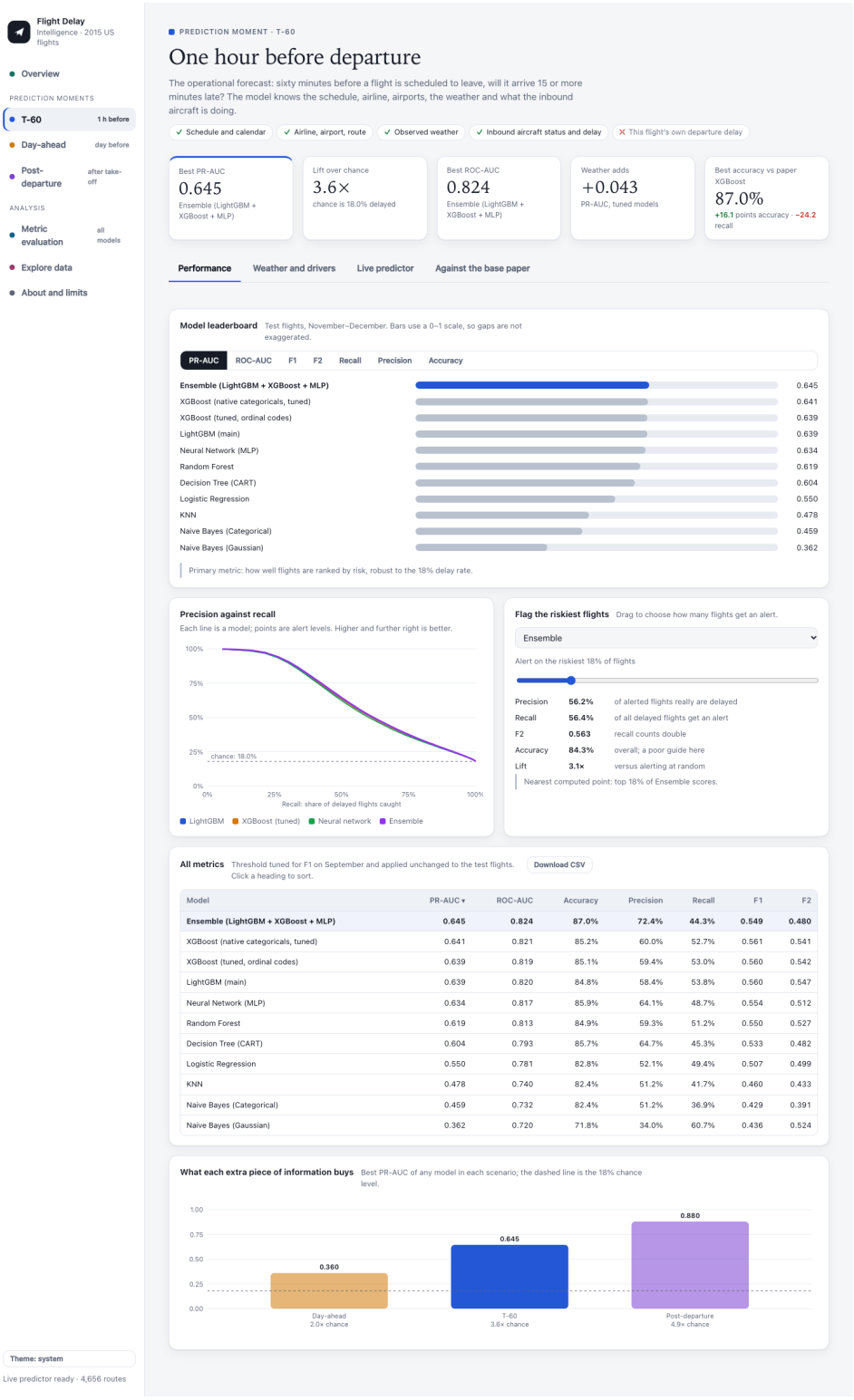


Figure 15: Portal: T-60 performance page

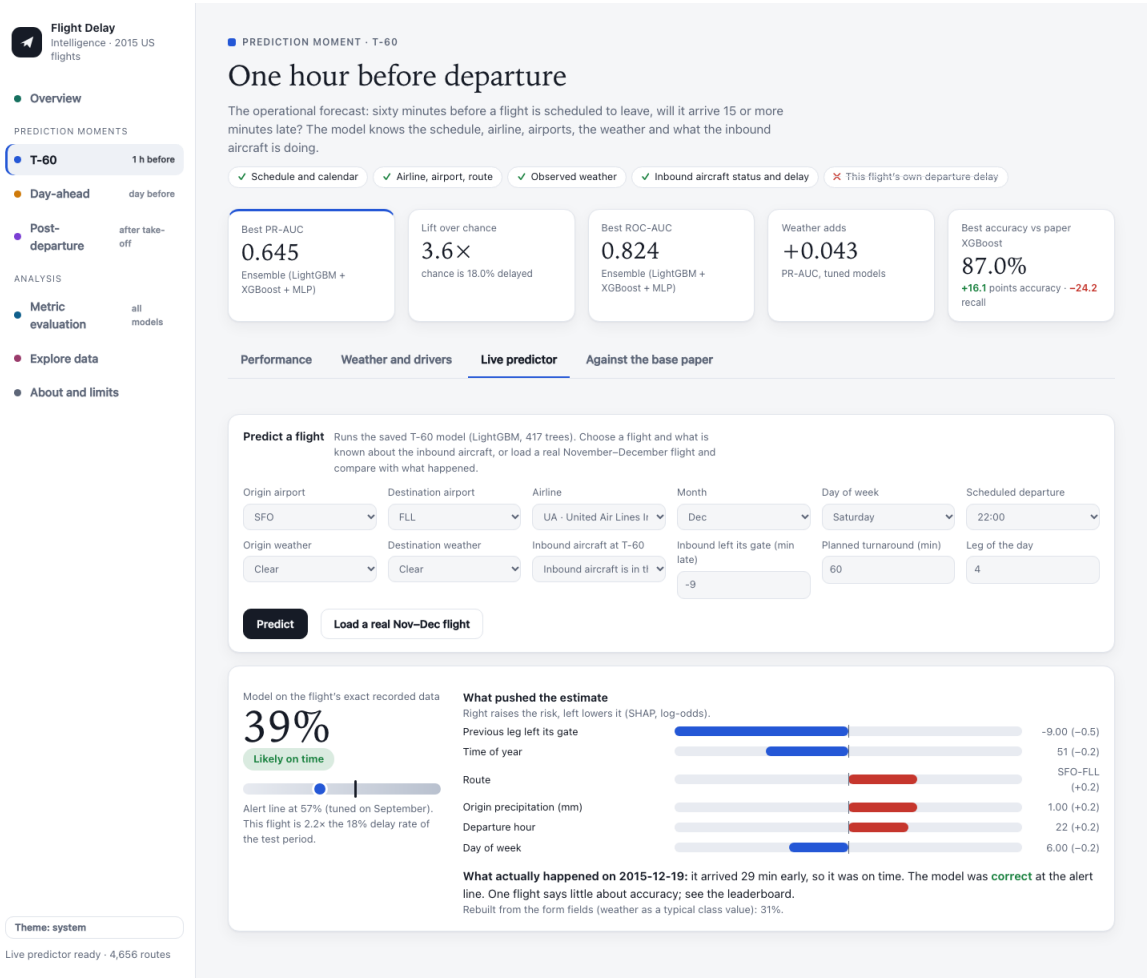


Figure 16: Portal: live predictor with a real flight

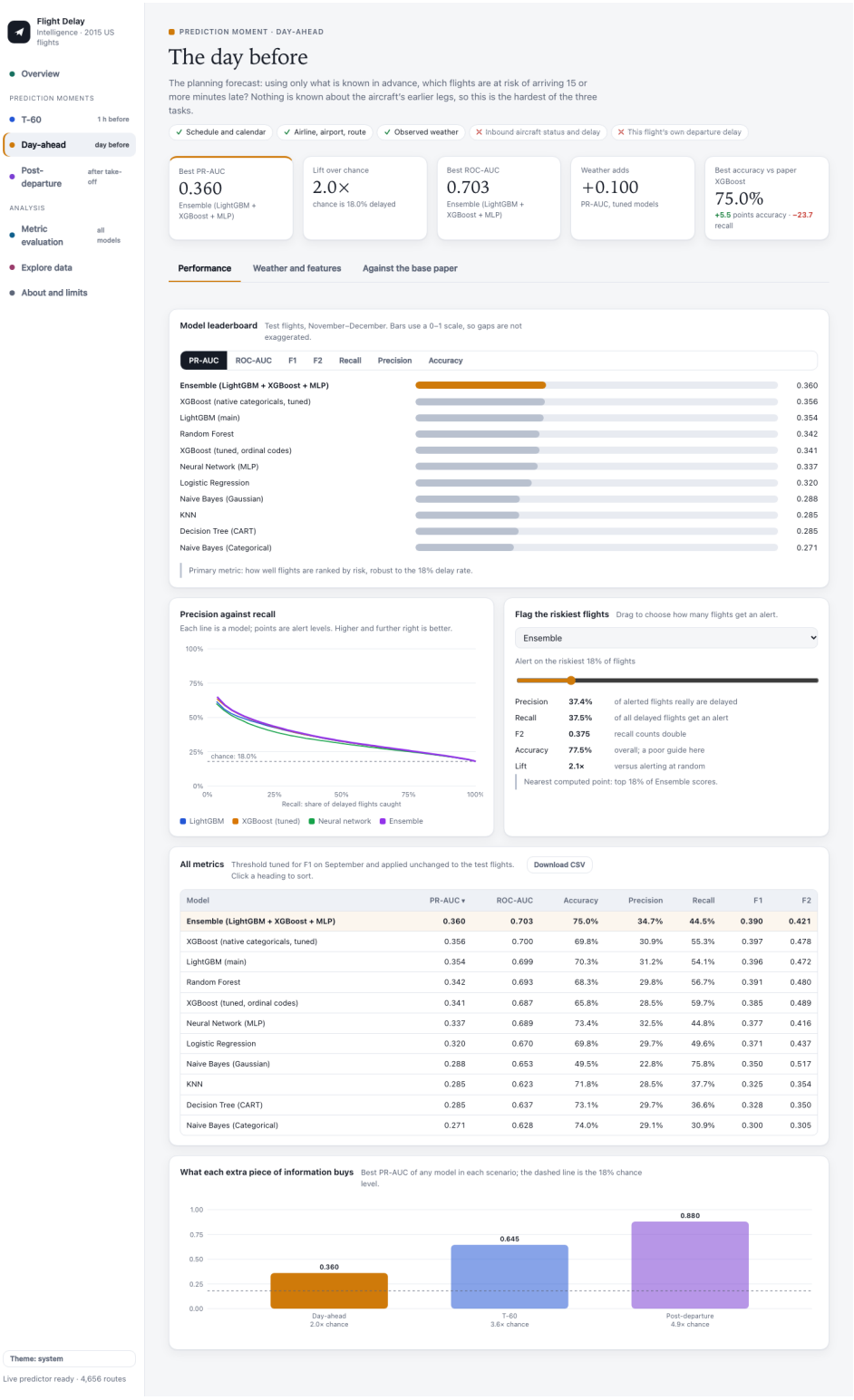


Figure 17: Portal: day-ahead performance page

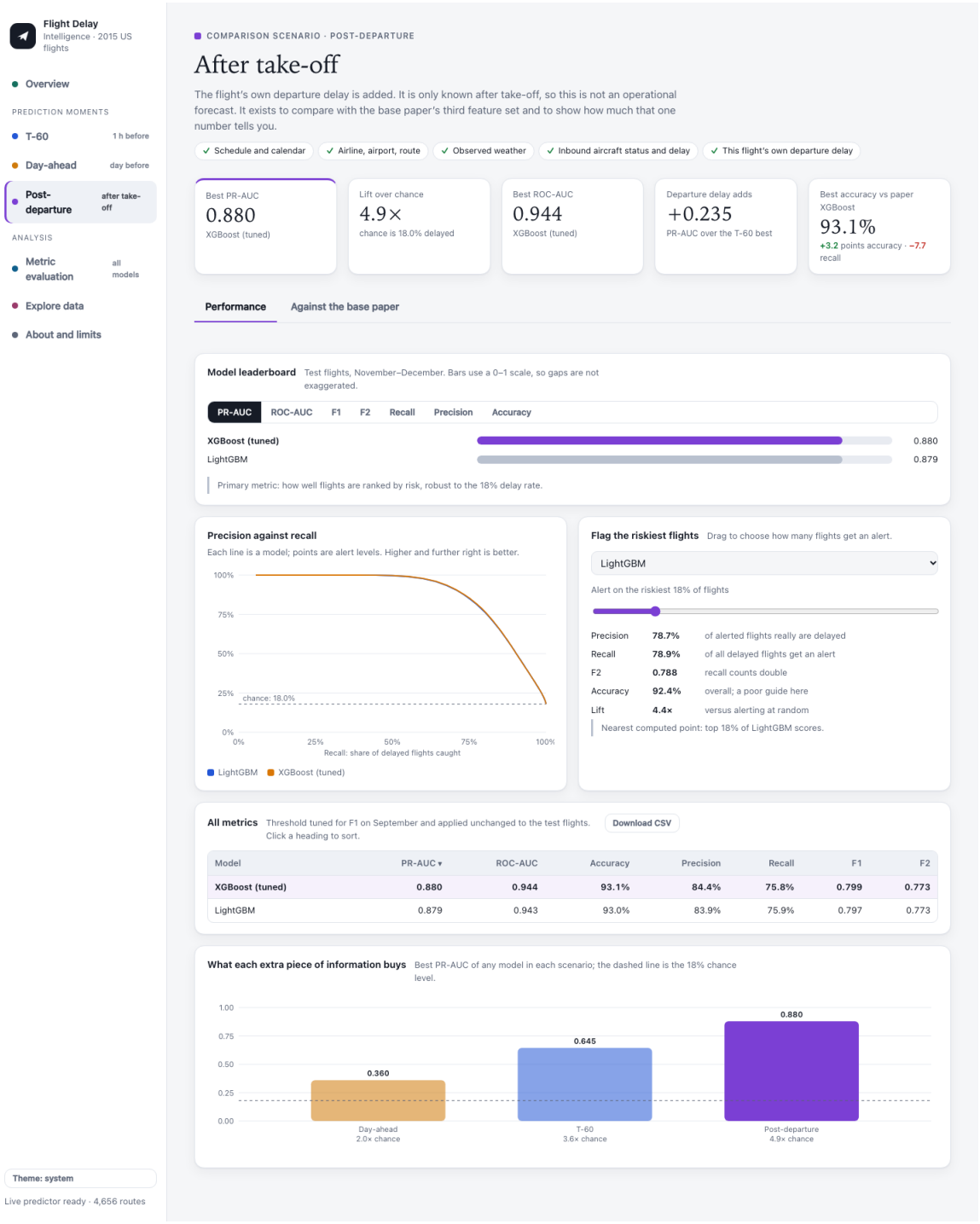


Figure 18: Portal: post-departure performance page

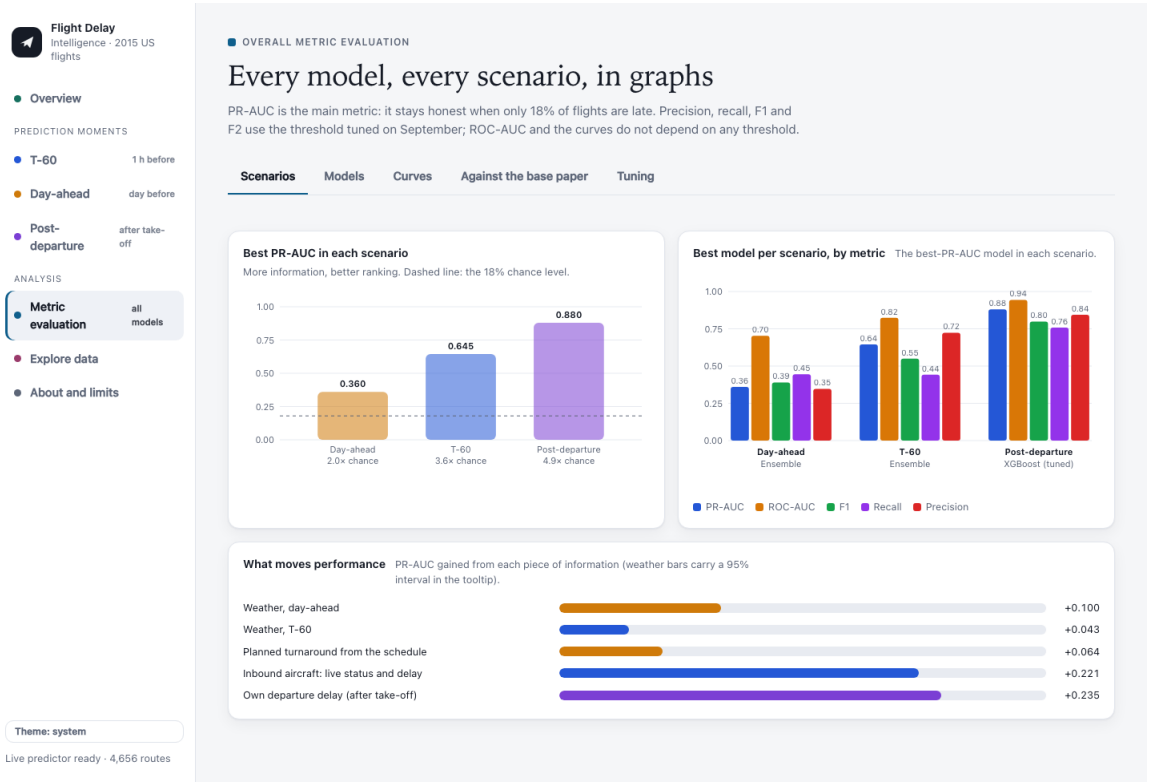


Figure 19: Portal: overall metric evaluation, scenarios

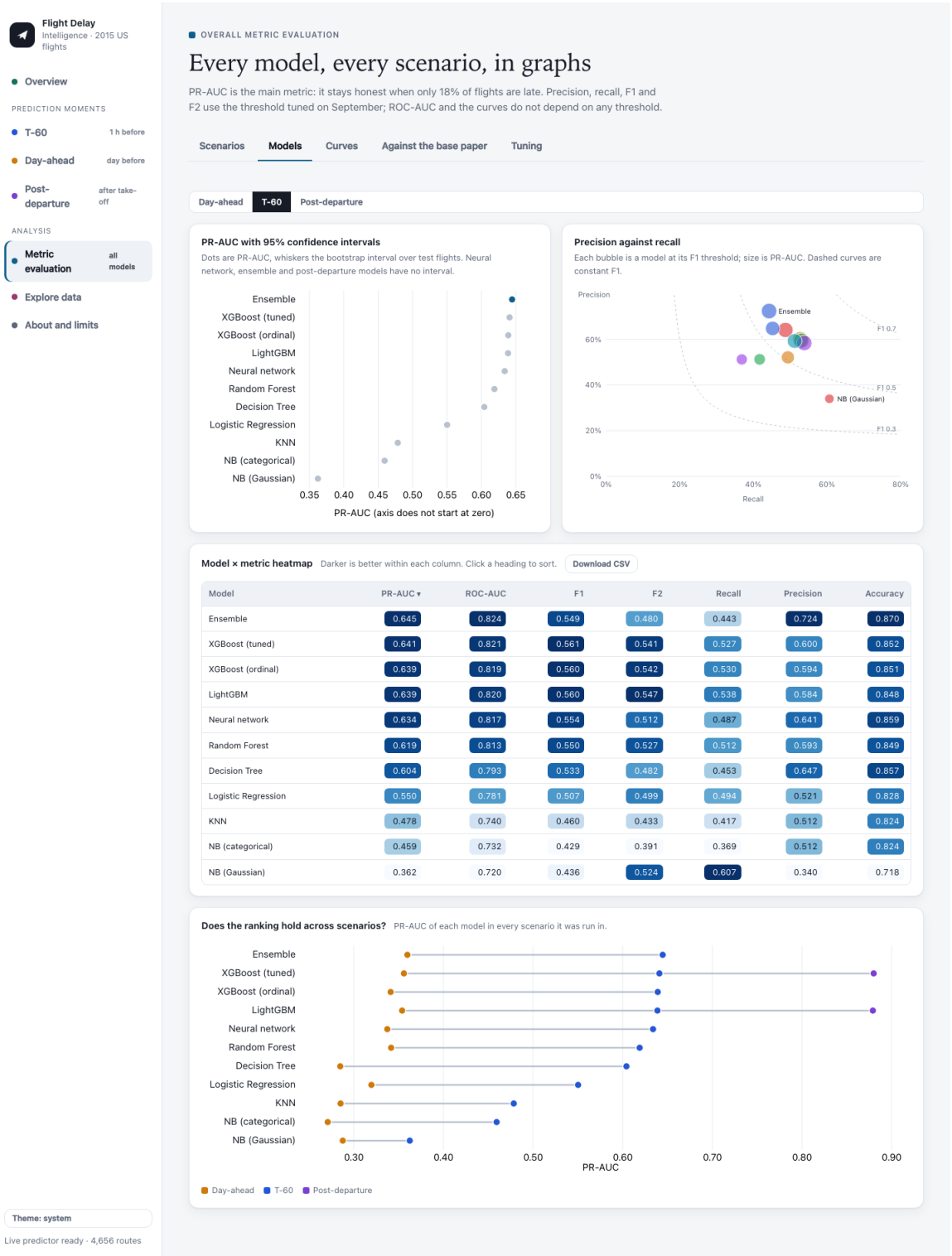


Figure 20: Portal: overall metric evaluation, models

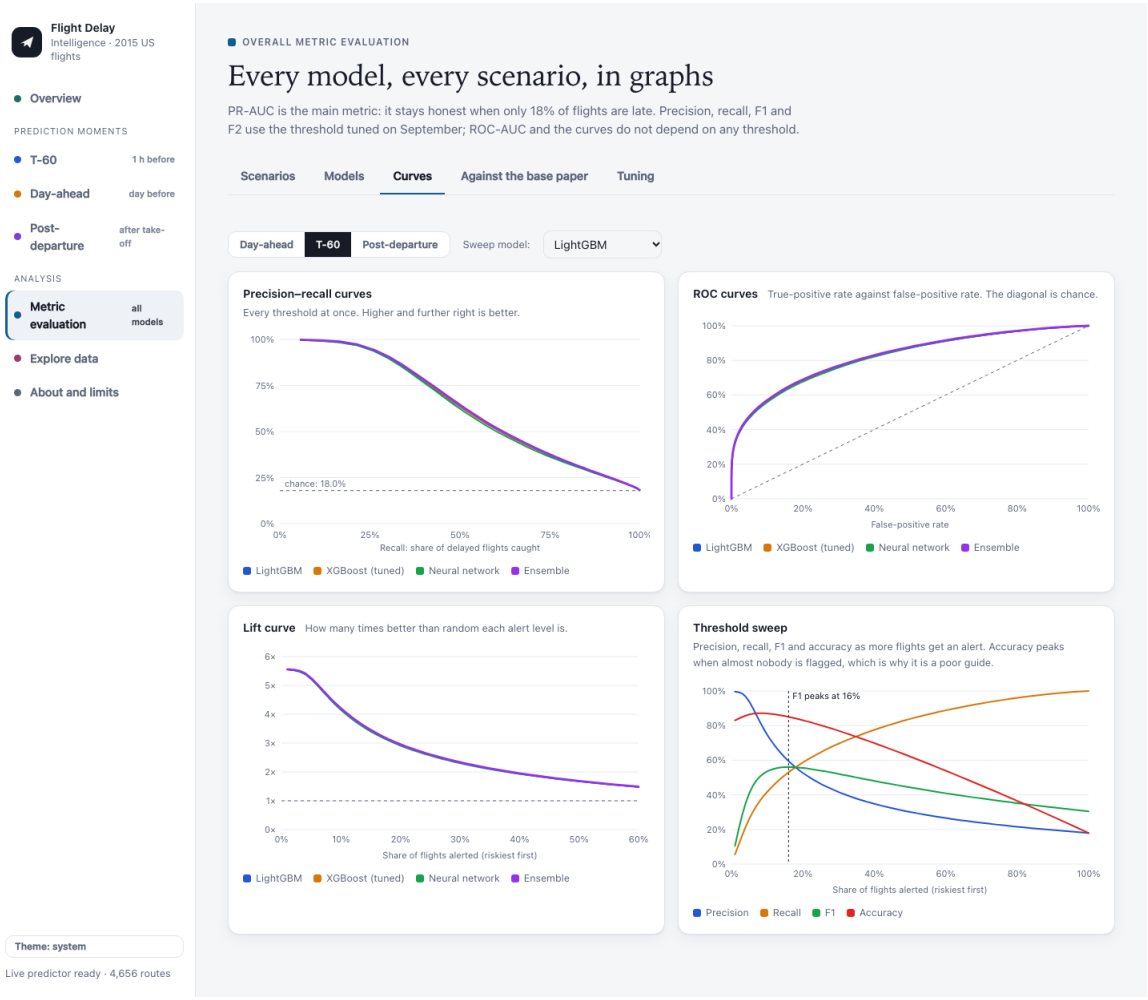


Figure 21: Portal: precision-recall, ROC and lift curves and the threshold sweep

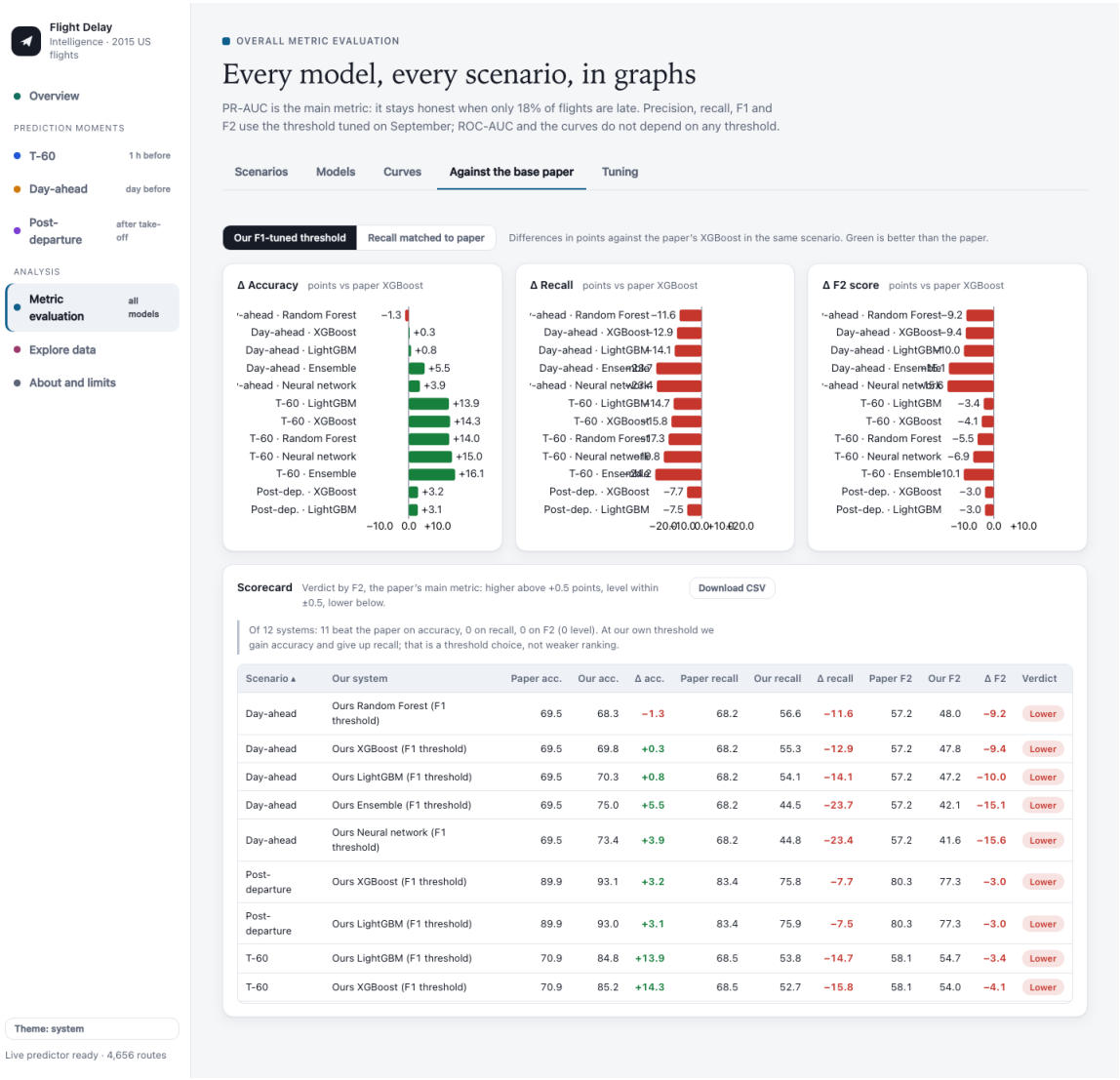


Figure 22: Portal: scorecard against the base paper

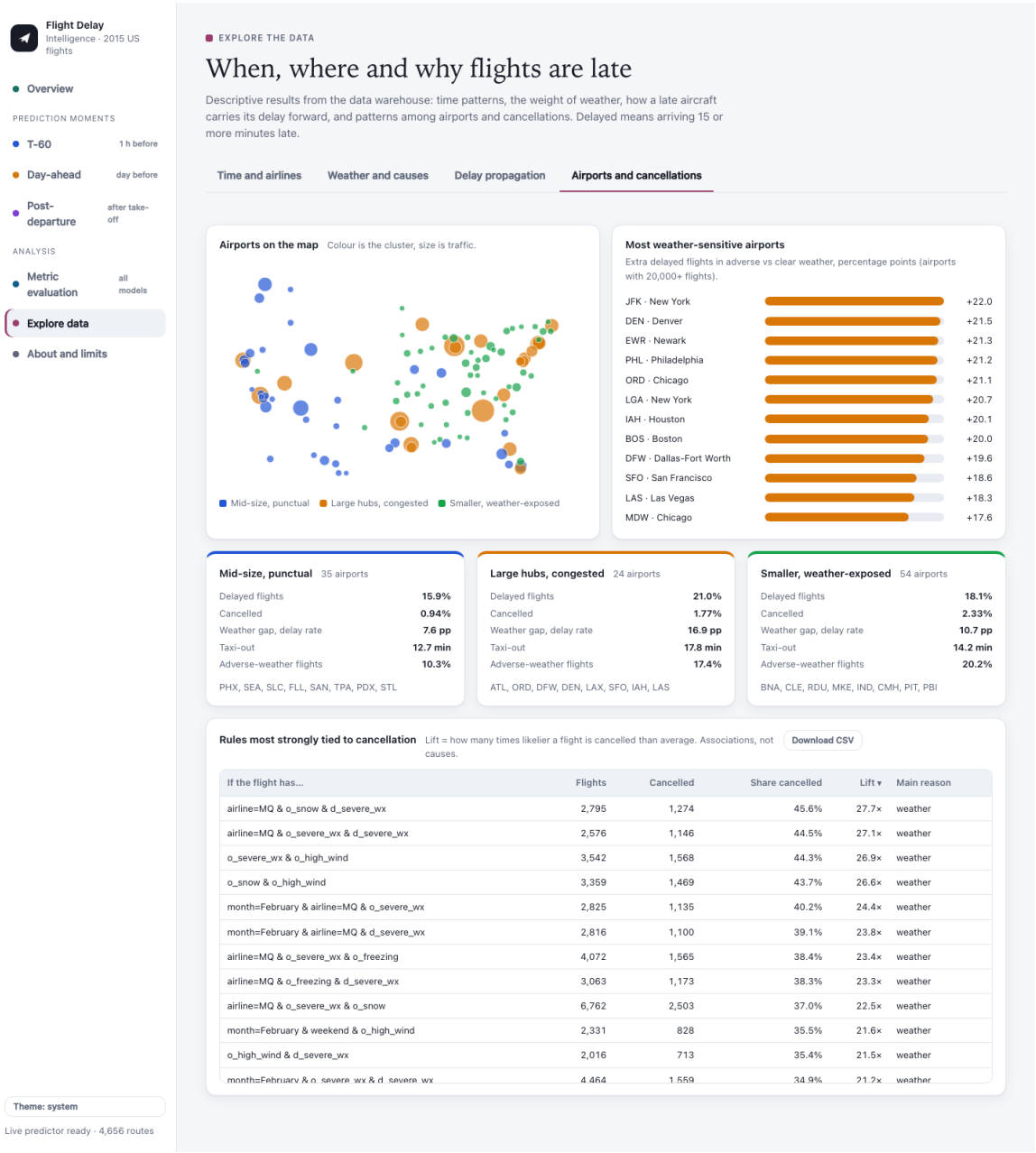


Figure 23: Portal: airports and cancellations

CHAPTER 7

REFERENCES

7.1 LIST OF JOURNALS

[1] Schösser D., Schönberger J. On the performance of machine learning based flight delay prediction - investigating the impact of short-term features. Promet - Traffic&Transportation. 2022;34(6):825-838.

[2] Gui G., Liu F., Sun J., Yang J., Zhou Z., Zhao D. Flight delay prediction based on aviation big data and machine learning. IEEE Transactions on Vehicular Technology. 2020;69(1):140-150.

[3] Yazdi M.F., Kamel S.R., Chabok S.J.M., Kheirabadi M. Flight delay prediction based on deep learning and Levenberg-Marquart algorithm. Journal of Big Data. 2020;7:106.

[4] Belcastro L., Marozzo F., Talia D., Trunfio P. Using scalable data mining for predicting flight delays. ACM Transactions on Intelligent Systems and Technology. 2016;8(1):1-20.

[5] Ke G., Meng Q., Finley T., et al. LightGBM: a highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems 30. 2017.

[6] Chen T., Guestrin C. XGBoost: a scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016:785-794.

[7] Lundberg S.M., Lee S.-I. A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems 30. 2017.

[8] Pedregosa F., Varoquaux G., Gramfort A., et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research. 2011;12:2825-2830.

[9] Menne M.J., Durre I., Vose R.S., Gleason B.E., Houston T.G. An overview of the Global Historical Climatology Network-Daily database. Journal of Atmospheric and Oceanic Technology. 2012;29(7):897-910.

[10] Raasveldt M., Mühleisen H. DuckDB: an embeddable analytical database. Proceedings of the 2019 International Conference on Management of Data (SIGMOD). 2019:1981-1984.

[11] Paszke A., Gross S., Massa F., et al. PyTorch: an imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems 32. 2019.

7.2 LIST OF WEBSITES (URLs)

[12] Bureau of Transportation Statistics. Airline on-time performance and causes of flight delays. https://www.bts.gov/topics/airlines-and-airports/airline-time-performance-and-causes-flight-delays

[13] U.S. Department of Transportation. 2015 Flight Delays and Cancellations (dataset). https://www.kaggle.com/datasets/usdot/flight-delays

[14] NOAA National Centers for Environmental Information. Global Historical Climatology Network - Daily (GHCN-Daily). https://www.ncei.noaa.gov/products/land-based-station/global-historical-climatology-network-daily

[15] DuckDB documentation. https://duckdb.org/docs/

[16] D3.js - Data-Driven Documents. https://d3js.org

[17] SHAP documentation. https://shap.readthedocs.io

7.3 LIST OF BOOKS

[18] Kimball R., Ross M. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. 3rd ed. Wiley; 2013.

[19] Han J., Kamber M., Pei J. Data Mining: Concepts and Techniques. 3rd ed. Morgan Kaufmann; 2011.

[20] Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning. 2nd ed. Springer; 2009.

[21] McKinney W. Python for Data Analysis. 3rd ed. O'Reilly Media; 2022.